

Employee Management System

Microservices-based web application

ASP.NET Core | PostgreSQL | Apache Kafka | Eureka | API Gateway | React

Version	1.0
Date	24 September 2026
Status	Issued for mentor review and development use
Prepared by	Employee Management System Project Team

1. Document Information

Item	Details
Document title	Software Requirements Specification (SRS)
Project	Employee Management System
Project type	Microservices-based web application (training / project application)
Document version	1.0
Status	Issued for mentor review and development use
Date	24 September 2026
Prepared by	Employee Management System Project Team
Reviewer	Project mentor
Source documents	Project requirements statement (source of truth); Employee_Management_System_Wireframe_Documentation.pdf (12 UI screens)
Classification	Internal project documentation

2. Revision History

Version	Date	Author	Description of change
1.0	24 September 2026	Project Team	First complete issue of the SRS covering all seven microservices, the supporting infrastructure, security, testing and frontend requirements.

3. Table of Contents

1. Document Information	2
2. Revision History	2
3. Table of Contents	3
4. Introduction	6
4.1 Document Conventions	6
4.2 Intended Audience	6
4.3 References	6
5. Purpose	6
6. Scope	7
6.1 In Scope	7
6.2 Out of Scope	7
7. Product Overview	7
7.1 Product Perspective	7
7.2 Major Capabilities	7
7.3 User Classes	7
8. Stakeholders	8
9. User Roles and Permissions	8
9.1 Role Definitions	8
9.2 Scope Definitions	8
9.3 Role Permission Matrix	8
9.4 Separation of Salary Permissions	10
10. System Overview	10
10.1 System Context	10
10.2 Technology Stack	10
10.3 Design Principles	10
11. System Architecture	11
11.1 Logical Architecture	11
11.2 Architecture Requirements	11
12. Microservice Responsibilities	12
12.1 Responsibilities by Service	12
12.2 Microservice Responsibility Matrix	12
13. Functional Requirements	13
13.1 Requirement Index	13
14. Detailed Requirements by Microservice	15
14.1 Conventions Applying to All Requirements	15

14.2 Authentication and User Management (Security Component)	15
14.3 Employee Service	17
14.4 Department Service	20
14.5 Salary Service	22
14.6 Leave Service	30
14.7 Attendance Service	33
14.8 Support Service	34
14.9 Notification Service	37
15. User Workflows	38
15.1 Use Case Summary	38
15.2 Major Workflows	38
16. Business Rules	39
16.1 General	39
16.2 Employee Service	40
16.3 Department Service	40
16.4 Salary Service	40
16.5 Leave Service	41
16.6 Attendance Service	41
16.7 Support Service	41
16.8 Notification Service	41
17. Data Requirements	42
18. Database and Normalization Requirements	45
18.1 Database Requirements	45
18.2 Normalization Requirements	45
18.3 Database Summary	46
19. API Requirements	46
19.1 General API Requirements	46
19.2 API Summary Table	47
20. Communication Requirements	49
20.1 Synchronous and Asynchronous Communication	49
20.2 Synchronous Service-to-Service Calls	49
21. Kafka / Event Requirements	50
21.1 Event Requirements	50
21.2 Kafka Event Summary	50
21.3 Event-to-Notification Mapping	50
22. Security Requirements	51
22.1 Authentication	51
22.2 Password Handling	51
22.3 Authorization	51
22.4 Sensitive Data and Secure API Access	51

23. API Gateway Requirements	52
24. Service Discovery Requirements	52
25. API Versioning Requirements	53
26. Central Logging Requirements	53
27. Non-Functional Requirements	53
27.1 Performance	53
27.2 Scalability	54
27.3 Availability	54
27.4 Reliability	54
27.5 Maintainability	54
27.6 Usability	54
27.7 Testability	54
27.8 Requirements Defined Elsewhere	55
28. Validation and Error Handling	55
28.1 Validation Rules	55
28.2 Error Handling Requirements	55
28.3 Error Catalogue	55
29. Testing Requirements	56
29.1 Test Types	56
29.2 Testing Requirements	56
29.3 Key Test Scenarios	56
30. Swagger / OpenAPI Requirements	58
31. Frontend Requirements	58
31.1 General Frontend Requirements	58
31.2 Wireframe Screen Mapping	58
32. Assumptions and Constraints	59
32.1 Constraints	59
32.2 Assumptions	59
32.3 Open Items	60
33. Future Enhancements	60
34. Acceptance Criteria	60
35. Glossary	61
Appendix A — Functional Requirement Traceability Matrix	62

4. Introduction

This document is the Software Requirements Specification for the Employee Management System, a microservices-based web application built as a training project. It specifies what the system shall do and the constraints under which it shall be built, in a form that can be used for development, testing and review.

4.1 Document Conventions

The words **shall** and **must** denote mandatory requirements; **should** denotes a recommended requirement; **may** denotes an optional capability. Every requirement carries a unique identifier that is used consistently in the traceability matrix.

Prefix	Meaning	Section
FR-xxx-nnn	Functional requirement (AUT, EMP, DEP, SAL, LEV, ATT, SUP, NOT)	14
BR-xxx-nnn	Business rule	16
ARC / COM / KAF / GW / SD / VER / DB / API / SWG / FE	Architecture, communication, Kafka, gateway, discovery, versioning, database, API, Swagger and frontend requirements	11, 18–26, 30, 31
NFR-xxx-nnn	Non-functional requirement (SEC, PERF, SCAL, AVL, MNT, REL, USB, LOG, ERR, TST)	22, 26, 27, 28
VAL / TR / TS / AC	Validation rule, testing requirement, test scenario, acceptance criterion	28, 29, 34
A-nn / OI-nn	Assumption / open item	32

4.2 Intended Audience

- Developers, who use it to implement the services, APIs and databases.
- Testers, who derive unit, API, integration, validation, authorization and error-handling tests from it.
- The project mentor, who reviews completeness, consistency and architectural correctness.
- Frontend developers, who use it together with the wireframe documentation.

4.3 References

- Project requirements statement for the Employee Management System (source of truth).
- Employee Management System — Wireframe Design Documentation, 12 screens (Employee_Management_System_Wireframe_Documentation.pdf).
- OpenAPI Specification; RFC 7807 (Problem Details for HTTP APIs) for the error-response structure.

5. Purpose

The purpose of this SRS is to define, in specific and testable terms, the functional, business, architectural, security, non-functional and testing requirements of the Employee Management System. It is intended to:

- give developers an unambiguous basis for building seven independent microservices and their supporting infrastructure;
- give testers requirements that can each be verified by a defined test type;
- give the mentor a single reference to review the project scope and design decisions;
- serve as the final project documentation.

Where the requirements statement is silent on a detail that is needed to make a requirement testable, the SRS records the choice as a numbered assumption or open item in Section 32 rather than presenting it as a stated requirement.

6. Scope

6.1 In Scope

The Employee Management System provides secure, role-based management of the following business areas through versioned REST APIs behind an API Gateway:

- Employee records and employee profiles (Employee Service);
- Departments (Department Service);
- Salary, salary components, salary revisions, bonuses, overtime, payroll and payslips (Salary Service);
- Leave requests and their approval workflow (Leave Service);
- Daily attendance (Attendance Service);
- Support tickets and their workflow (Support Service);
- Notifications generated from asynchronous events (Notification Service);
- Supporting infrastructure: API Gateway, Eureka service discovery, Apache Kafka, PostgreSQL, central logging, security (authentication and authorization) and API versioning;
- A React frontend, if implemented, following the supplied wireframes.

6.2 Out of Scope

- A Project Service or any project-management capability. It is not part of the current project.
- Any feature not described in this document (see Section 33 for possible future enhancements).
- Delivery of notifications through channels other than in-app notifications (for example SMS or push).

7. Product Overview

7.1 Product Perspective

The system is a new, self-contained product. Clients (the React web application, Swagger UI and Bruno) communicate only with the API Gateway. The gateway routes requests to seven business microservices. Each microservice owns its own PostgreSQL database. Synchronous calls between services use REST; asynchronous business events use Apache Kafka. Services register with Eureka, write to central logging, and rely on the security component for authentication.

7.2 Major Capabilities

Area	Capabilities
Employee Service	Add, View, View All, Search, Update and Delete employees; own-profile view and edit; team view.
Department Service	Add, View, View All, Update and Delete departments.
Salary Service	Salary, salary components, salary revision, bonus, overtime request and approval, payroll, payslip.
Leave Service	Apply, View, View History, Approve and Reject leave.
Attendance Service	Check In, Check Out, View Attendance, View History, View Team Attendance.
Support Service	Create, View, Assign, Update Status, Resolve and Close tickets.
Notification Service	Receive Kafka events, create notifications, view notifications, mark as read.

7.3 User Classes

Five roles use the system: Admin, HR, Manager, Employee and Support Staff. They are defined in Section 9.

8. Stakeholders

Stakeholder	Interest in the system
Project mentor / reviewer	Reviews requirements, architecture, security and testing approach; evaluates the final project.
Development team	Implements services, APIs, databases, messaging and frontend.
Test team	Designs and executes unit, API, integration, validation, authorization and error-handling tests.
Admin	Administers employees, departments, salary and payroll, users and roles, and support operations.
HR	Maintains employee and department data, salary, bonuses, overtime approvals, payroll, payslips and leave.
Managers	Oversee team members, approve leave and overtime, view team attendance.
Employees	Use self-service functions: profile, salary view, leave, attendance, overtime, support and notifications.
Support Staff	Handle and resolve support tickets.

9. User Roles and Permissions

9.1 Role Definitions

Role	Responsibilities
Admin	Manage employees; manage departments; manage salary and payroll; manage users and roles; manage support operations; access administrative information.
HR	Add, view, search, update and delete employees; add, view, update and delete departments; manage salary; add salary components; create salary revisions; add bonuses; review and approve overtime; generate payroll; generate payslips; manage leave operations.
Manager	View team employees; view team attendance; view leave requests; approve or reject leave; view overtime requests; approve or reject overtime; perform authorized support operations.
Employee	Login; view and edit own profile; view salary, salary revision history, approved bonus, approved overtime and payslip; apply leave; view leave history; check in and out; view attendance; submit overtime requests; create and view own support tickets; view notifications and mark them as read.
Support Staff	View support tickets and ticket details; assign tickets; update ticket status; resolve tickets; close tickets; view notifications.

9.2 Scope Definitions

Scope	Meaning
All	Every record of the resource type.
Team	Records of employees who belong to the Manager's team (Assumption A-03).
Own	Records whose EmployeeId equals the EmployeeId in the actor's access token.
Assigned	Support tickets whose AssignedTo equals the actor's EmployeeId.
(R)	Read-only access.

Every authenticated user, whatever their role, holds the self-service ("Own") permissions listed for the Employee role (BR-GEN-001). Additional permissions come from the role.

9.3 Role Permission Matrix

Capability	Admin	HR	Manager	Employee	Support Staff
Authentication and users					
Login; request and complete password reset	All	All	All	All	All
Manage user accounts and roles	All	—	—	—	—
Employees					
Add employee	All	All	—	—	—

Capability	Admin	HR	Manager	Employee	Support Staff
View employee record	All	All	Team	Own	Own
View all employees	All	All	–	–	–
Search employees	All	All	Team	–	–
View team employees	–	–	Team	–	–
Update employee record	All	All	–	–	–
View / edit own profile (limited fields)	Own	Own	Own	Own	Own
Delete employee	All	All	–	–	–
Departments					
View departments	All	All	All	All	All
Add / update / delete department	All	All	–	–	–
Salary, bonus and revision					
Add / update salary	All	All	–	–	–
View salary	All	All	Own	Own	Own
Add / update salary components	All	All	–	–	–
View salary components	All	All	Own	Own	Own
Create salary revision	All	All	–	–	–
View salary revision history	All	All	Own	Own	Own
Add / update bonus	All	All	–	–	–
View bonus	All	All	Own (Approved)	Own (Approved)	Own (Approved)
Overtime					
Submit overtime request	Own	Own	Own	Own	Own
View overtime requests / approved overtime	All	All	Team + own	Own	Own
Approve / reject overtime	All	All	Team	–	–
Include approved overtime in payroll	All	All	–	–	–
Payroll and payslip					
Generate payroll; view payroll and payroll items	All	All	–	–	–
Generate payslips	All	All	–	–	–
View payslip	All	All	Own	Own	Own
Leave					
Apply leave; view own leave and history	Own	Own	Own	Own	Own
View leave requests (review list)	All (R)	All	Team	–	–
Approve / reject leave	–	All	Team	–	–
Attendance					
Check in / check out; view own attendance and history	Own	Own	Own	Own	Own
View team attendance	All	All	Team	–	–
Support					
Create ticket; view own tickets	Own	Own	Own	Own	Own
View all tickets and ticket details	All	–	–	–	All
Assign ticket	All	–	–	–	All
Update status, resolve, close ticket	All	–	–	–	Assigned
Notifications					
View notifications; mark as read	Own	Own	Own	Own	Own

Managers may create and view their own tickets. Any further support operations "authorized" for Managers are to be defined by Admin (Open Item OI-04); none are assumed by this document.

9.4 Separation of Salary Permissions

Salary, bonus, overtime and salary revision permissions are deliberately separated. Employees never create or modify salary, bonus or salary revision data.

Data	Employee (and Support Staff)	Manager	HR	Admin
Salary	View own	View own only	Add, update, view all	Add, update, view all
Salary components	View own	View own only	Add, update, view all	Add, update, view all
Salary revision	View own history	View own history	Create, view all	Create, view all
Bonus	View own Approved	View own Approved	Add, update, view all	Add, update, view all
Overtime request	Submit, view own	Submit, view own; approve or reject team	View all; approve or reject; include in payroll	View all; approve or reject; include in payroll
Payroll and items	No access	No access	Generate, view	Generate, view
Payslip	View own	View own	Generate, view all	Generate, view all

10. System Overview

10.1 System Context

Users interact through a browser (React frontend) or through API tools. All requests pass through the API Gateway, which validates the access token and routes the request to the responsible microservice. The system has no other external actors; e-mail dispatch for password reset is the only outbound integration and is addressed by Open Item OI-05.

10.2 Technology Stack

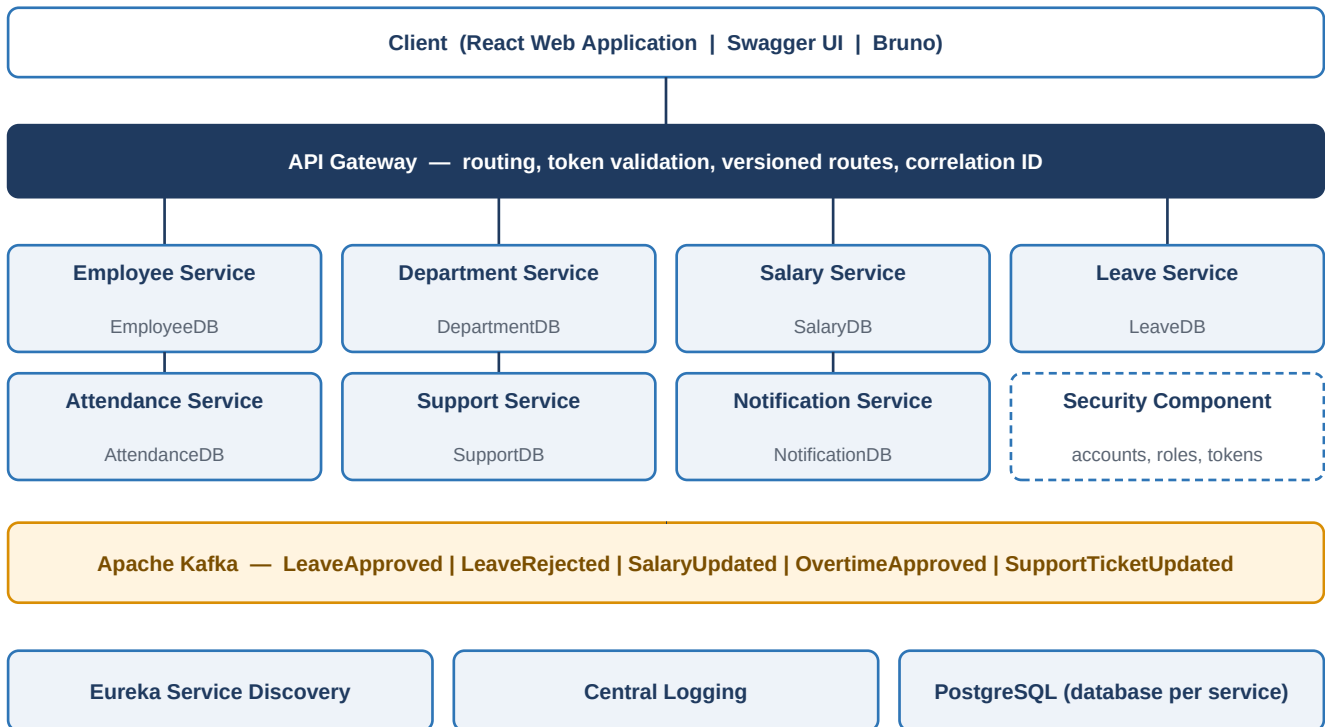
Concern	Technology
Backend	C#, .NET, ASP.NET Core Web API
Database	PostgreSQL (one database per service)
Data access	Entity Framework Core
API documentation	Swagger / OpenAPI
API testing	Bruno
Unit testing	xUnit
Frontend	React (if implemented)
Messaging	Apache Kafka
Service discovery	Eureka
Edge	API Gateway (required)
Security	Authentication and authorization, role-based access control
Cross-cutting	API versioning (required); central logging (required)

10.3 Design Principles

- Each service owns its data and business rules; cross-service data is referenced by identifier only.
- REST for synchronous operations that need an immediate response; Kafka only for asynchronous events.
- Least privilege: every operation is authorized by role and, where relevant, by ownership or team membership.
- Consistent naming across requirements, APIs, databases and services (EmployeeId, DepartmentId, SalaryId, LeaveId, AttendancId, TicketId, NotificationId).

11. System Architecture

11.1 Logical Architecture



Solid boxes: business microservices and infrastructure. Dashed box: supporting security component (not a business microservice).

REST (synchronous) flows Client > API Gateway > Service, and Service > Service where an immediate response is required. Kafka carries asynchronous events only.

Figure 1 — Logical architecture: Client > API Gateway > Microservices, with supporting infrastructure.

11.2 Architecture Requirements

ID	Requirement
ARC-001	The system shall be implemented as a microservices architecture consisting of exactly seven business microservices: Employee, Department, Salary, Leave, Attendance, Support and Notification. No Project Service shall exist.
ARC-002	Clients shall reach services only through the API Gateway (Client > API Gateway > Microservices). Services shall not be exposed directly to clients.
ARC-003	Each microservice shall be an ASP.NET Core Web API written in C# on .NET, using Entity Framework Core to access its own PostgreSQL database.
ARC-004	The system shall use database-per-service: Employee Service > EmployeeDB, Department Service > DepartmentDB, Salary Service > SalaryDB, Leave Service > LeaveDB, Attendance Service > AttendanceDB, Support Service > SupportDB, Notification Service > NotificationDB.
ARC-005	A service shall not read or write another service's database. Data owned by another service shall be obtained through its REST API or from Kafka events.
ARC-006	REST shall be used for synchronous operations that need an immediate response. Kafka shall be used only for asynchronous events.
ARC-007	Every service shall register with Eureka and shall be located through the registry rather than through hard-coded addresses.
ARC-008	Each service shall be independently buildable, testable, configurable and deployable.
ARC-009	Each service shall separate API (controller), business logic and data-access responsibilities.
ARC-010	Services shall be stateless; all persistent state shall reside in the service's database.

ID	Requirement
ARC-011	Cross-cutting concerns (logging, error format, token handling) may be shared as libraries; business rules shall not be shared between services.
ARC-012	Environment-specific configuration and secrets shall be supplied outside the source code.
ARC-013	The security component shall provide authentication and role assignment. It is supporting infrastructure, is not a business microservice, and its credential store shall not be accessible to any other service.

12. Microservice Responsibilities

12.1 Responsibilities by Service

Service	Responsible for	Not responsible for
Employee Service	Employee records; unique Email; own-profile maintenance; team membership lookup; employee-count lookup per DepartmentId.	Department data, salary data, leave, attendance, tickets, notifications.
Department Service	Department records; unique DepartmentName; blocking deletion of departments that still have employees.	Employee records.
Salary Service	Salary, components, revisions, bonuses, overtime requests and approvals, payroll, payroll items, payslips; publishing SalaryUpdated and OvertimeApproved.	Employee master data; leave; attendance.
Leave Service	Leave requests, status workflow, approval decisions; publishing LeaveApproved and LeaveRejected.	Attendance records; salary impact of leave.
Attendance Service	Check-in, check-out, attendance history, team attendance view.	Leave, overtime and payroll.
Support Service	Tickets, assignment, status workflow; publishing SupportTicketUpdated.	Employee master data; notification storage.
Notification Service	Consuming Kafka events; creating, listing and marking notifications.	Originating business data; creating notifications through any path other than events.

12.2 Microservice Responsibility Matrix

Service	Owned database	Calls (REST)	Publishes (Kafka)	Consumes (Kafka)
Employee	EmployeeDB	Department Service (validate DepartmentId)	–	–
Department	DepartmentDB	Employee Service (count employees per DepartmentId)	–	–
Salary	SalaryDB	Employee Service (validate EmployeeId, team scope, EmploymentStatus)	SalaryUpdated, OvertimeApproved	–
Leave	LeaveDB	Employee Service (team scope)	LeaveApproved, LeaveRejected	–
Attendance	AttendanceDB	Employee Service (team scope)	–	–
Support	SupportDB	Employee Service (validate assignee Role)	SupportTicketUpdated	–
Notification	NotificationDB	–	–	LeaveApproved, LeaveRejected, SalaryUpdated, OvertimeApproved, SupportTicketUpdated
Security component	Own credential store	Employee Service (validate EmployeeId when creating accounts)	–	–

Self-service operations take the caller's EmployeeId from the access token and therefore need no REST call to identify the caller; the calls listed above are only for data the token does not carry.

13. Functional Requirements

This section lists all functional requirements. Section 14 specifies each requirement in detail. Identifiers follow the pattern FR-<SERVICE>-<NNN>; the authentication requirements use the code AUT and are delivered by the supporting Security component.

13.1 Requirement Index

ID	Requirement	Service	Access
FR-AUT-001	Login	Security Component	Anonymous
FR-AUT-002	Request Password Reset	Security Component	Anonymous
FR-AUT-003	Reset Password	Security Component	Anonymous
FR-AUT-004	Manage User Accounts	Security Component	Admin
FR-AUT-005	Assign User Role	Security Component	Admin
FR-EMP-001	Add Employee	Employee	HR, Admin
FR-EMP-002	View Employee	Employee	HR, Admin: all. Manager: team. Others: own
FR-EMP-003	View All Employees	Employee	HR, Admin
FR-EMP-004	Search Employees	Employee	HR, Admin: all. Manager: team
FR-EMP-005	Update Employee	Employee	HR, Admin
FR-EMP-006	Delete Employee	Employee	HR, Admin
FR-EMP-007	View Own Profile	Employee	All roles (own data)
FR-EMP-008	Update Own Profile	Employee	All roles (own data)
FR-EMP-009	View Team Employees	Employee	Manager
FR-DEP-001	Add Department	Department	HR, Admin
FR-DEP-002	View Department	Department	All authenticated roles
FR-DEP-003	View All Departments	Department	All authenticated roles
FR-DEP-004	Update Department	Department	HR, Admin
FR-DEP-005	Delete Department	Department	HR, Admin
FR-SAL-001	Add Salary	Salary	HR, Admin
FR-SAL-002	View Salary	Salary	HR, Admin: all. Others: own
FR-SAL-003	Update Salary	Salary	HR, Admin
FR-SAL-004	Add Salary Component	Salary	HR, Admin
FR-SAL-005	Update Salary Component	Salary	HR, Admin
FR-SAL-006	View Salary Components	Salary	HR, Admin: all. Others: own
FR-SAL-007	Create Salary Revision	Salary	HR, Admin
FR-SAL-008	View Salary Revision History	Salary	HR, Admin: all. Others: own
FR-SAL-009	Add Bonus	Salary	HR, Admin
FR-SAL-010	Update Bonus	Salary	HR, Admin
FR-SAL-011	View Bonus	Salary	HR, Admin: all. Others: own, Approved only
FR-SAL-012	Submit Overtime Request	Salary	All roles (own data)
FR-SAL-013	View Overtime Requests	Salary	HR, Admin: all. Manager: team. Others: own
FR-SAL-014	Approve Overtime	Salary	Manager: team. HR, Admin
FR-SAL-015	Reject Overtime	Salary	Manager: team. HR, Admin
FR-SAL-016	Include Approved Overtime in Payroll	Salary	HR, Admin
FR-SAL-017	Generate Payroll	Salary	HR, Admin
FR-SAL-018	View Payroll	Salary	HR, Admin

ID	Requirement	Service	Access
FR-SAL-019	View Payroll Items	Salary	HR, Admin
FR-SAL-020	Generate Payslip	Salary	HR, Admin
FR-SAL-021	View Payslip	Salary	HR, Admin: all. Others: own
FR-LEV-001	Apply Leave	Leave	All roles (own data)
FR-LEV-002	View Leave	Leave	HR, Admin: all. Manager: team. Others: own
FR-LEV-003	View Leave History	Leave	All roles (own data)
FR-LEV-004	View Leave Requests	Leave	HR, Admin: all. Manager: team
FR-LEV-005	Approve Leave	Leave	Manager: team. HR
FR-LEV-006	Reject Leave	Leave	Manager: team. HR
FR-ATT-001	Check In	Attendance	All roles (own data)
FR-ATT-002	Check Out	Attendance	All roles (own data)
FR-ATT-003	View Attendance	Attendance	All roles (own data)
FR-ATT-004	View Attendance History	Attendance	All roles (own data)
FR-ATT-005	View Team Attendance	Attendance	HR, Admin: all. Manager: team
FR-SUP-001	Create Support Ticket	Support	All roles (own data)
FR-SUP-002	View Ticket	Support	Support Staff, Admin: all. Others: own
FR-SUP-003	View Tickets	Support	Support Staff, Admin: all. Others: own
FR-SUP-004	Assign Ticket	Support	Support Staff, Admin
FR-SUP-005	Update Ticket Status	Support	Support Staff (assignee), Admin
FR-SUP-006	Resolve Ticket	Support	Support Staff (assignee), Admin
FR-SUP-007	Close Ticket	Support	Support Staff (assignee), Admin
FR-NOT-001	Create Notification from Event	Notification	System
FR-NOT-002	View Notifications	Notification	All roles (own data)
FR-NOT-003	Mark Notification as Read	Notification	All roles (own data)

Total: 61 functional requirements.

14. Detailed Requirements by Microservice

14.1 Conventions Applying to All Requirements

- All endpoints are versioned under /api/v1 and are reached through the API Gateway.
- Unless a requirement is marked Anonymous or System, an unauthenticated request receives 401 UNAUTHENTICATED and a request whose role or scope is not permitted receives 403 FORBIDDEN. These two conditions are added to every such requirement.
- List endpoints accept pageNumber (default 1) and pageSize (default 20, maximum 100) and return totalCount.
- Identifiers are UUIDs; dates use ISO 8601; timestamps are UTC.
- Error responses follow the structure and codes in Section 28.

14.2 Authentication and User Management (Security Component)

These requirements are delivered by the supporting Security component. It is not a business microservice; it owns user accounts, credentials and role assignments.

FR-AUT-001 — Login	
Description	Allow a registered user to authenticate with credentials and obtain an access token for subsequent API calls.
Actor	Any registered user
Preconditions	An active user account exists for the user.
Main Flow	1. User submits login email and password. 2. System validates that both values are present and well-formed. 3. System verifies the password against the stored password hash. 4. System issues a signed access token containing UserId, EmployeeId, role and expiry. 5. System returns the token and its expiry time.
Validation / Business Rules	BR-GEN-001. Passwords are compared by hash only. Failure message is generic and does not reveal whether the email or password was wrong. Deactivated accounts cannot log in. Account is temporarily locked after 5 consecutive failed attempts (configurable). The role selector shown on the login wireframe is informational only and is never used to grant access.
Expected Result	HTTP 200 with access token and expiry. The login attempt is logged without credentials or token.
Error Conditions	• Missing or malformed fields: 400 VALIDATION_ERROR • Wrong credentials, deactivated or locked account: 401 UNAUTHENTICATED (generic message)
API Endpoint(s)	POST /api/v1/auth/login

FR-AUT-002 — Request Password Reset	
Description	Allow a user who has forgotten a password to request a reset link using the registered email.
Actor	Any user (unauthenticated)
Preconditions	None.
Main Flow	1. User enters the registered email on the Forgot Password screen. 2. System validates the email format. 3. System returns a generic confirmation regardless of whether the email is registered. 4. If the email belongs to an active account, system generates a single-use reset token valid for 30 minutes and dispatches a reset link to that email.
Validation / Business Rules	Responses must not allow user enumeration. Reset tokens are stored only as hashes. Earlier unused tokens of the same account are invalidated. The dispatch mechanism is recorded as Open Item OI-05.
Expected Result	HTTP 202 Accepted with a generic message.
Error Conditions	• Malformed email: 400 VALIDATION_ERROR • Dispatch failure: logged internally and not disclosed to the caller
API Endpoint(s)	POST /api/v1/auth/forgot-password

FR-AUT-003 — Reset Password	
Description	Allow a user to set a new password using a valid reset token.
Actor	Any user holding a reset token
Preconditions	A reset token was issued by FR-AUT-002 and has not expired or been used.
Main Flow	1. User opens the reset link and submits token, new password and confirmation. 2. System validates the token (exists, unexpired, unused). 3. System validates the new password against the password policy. 4. System stores the new password hash and marks the token as used.
Validation / Business Rules	Password policy: minimum 8 characters containing upper-case, lower-case, digit and special character. New password and confirmation must match. Token is single-use (NFR-SEC-004).
Expected Result	HTTP 204. The user can log in with the new password.
Error Conditions	• Invalid, expired or used token: 400 VALIDATION_ERROR • Policy failure or mismatch: 400 VALIDATION_ERROR
API Endpoint(s)	POST /api/v1/auth/reset-password

FR-AUT-004 — Manage User Accounts	
Description	Allow Admin to Create, View, Update and Deactivate user accounts.
Actor	Admin
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Admin submits EmployeeId, login email and initial role to create an account. 2. System verifies through Employee Service (REST) that the EmployeeId exists. 3. System creates the account in Active state; the user sets a password through FR-AUT-002 and FR-AUT-003. 4. Admin can list and view accounts, update the login email, or deactivate an account.
Validation / Business Rules	BR-GEN-001. One account per EmployeeId. Login email is unique. Deactivation keeps the account record. Admin cannot deactivate their own account.
Expected Result	201 Created (create); 200 (view, update); 204 (deactivate).
Error Conditions	• Employee not found: 400 REFERENCE_NOT_FOUND • Duplicate email or existing account: 409 CONFLICT • Invalid data: 400 VALIDATION_ERROR • Employee Service unavailable: 503 DEPENDENCY_UNAVAILABLE • Account not found: 404 NOT_FOUND • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	POST /api/v1/users GET /api/v1/users GET /api/v1/users/{userId} PUT /api/v1/users/{userId} PUT /api/v1/users/{userId}/deactivate

FR-AUT-005 — Assign User Role	
Description	Allow Admin to assign or change the role of a user account.
Actor	Admin
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Admin selects a user account and a role. 2. System validates that the role is one of Admin, HR, Manager, Employee or SupportStaff. 3. System saves the role; it takes effect in access tokens issued after the change.
Validation / Business Rules	BR-EMP-007. At least one active Admin account must remain. The role value must be kept consistent with Employee.Role (Open Item OI-02).
Expected Result	HTTP 200 with the updated account.
Error Conditions	• Invalid role: 400 VALIDATION_ERROR • Removing the last active Admin: 409 CONFLICT • Account not found: 404 NOT_FOUND • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	PUT /api/v1/users/{userId}/role

14.3 Employee Service

Service base path: /api/v1/employees. Database: EmployeeDB.

FR-EMP-001 — Add Employee	
Description	Create a new employee record.
Actor	HR, Admin
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor submits FirstName, LastName, Email, Phone, DateOfJoining, DepartmentId, Role and EmploymentStatus. 2. System validates required fields and formats. 3. System verifies that Email is not used by another employee. 4. System verifies through Department Service (REST) that DepartmentId exists. 5. System generates EmployeeId, sets CreatedAt and UpdatedAt, and stores the record.
Validation / Business Rules	BR-EMP-001, BR-EMP-002, BR-EMP-003, BR-EMP-004, BR-EMP-007, BR-EMP-008, BR-GEN-004.
Expected Result	HTTP 201 with the new EmployeeId and the resource location.
Error Conditions	• Missing or invalid field: 400 VALIDATION_ERROR (field-level details) • Duplicate Email: 409 CONFLICT • Unknown DepartmentId: 400 REFERENCE_NOT_FOUND • Department Service unavailable: 503 DEPENDENCY_UNAVAILABLE • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	POST /api/v1/employees

FR-EMP-002 — View Employee	
Description	Retrieve one employee record by EmployeeId.
Actor	HR, Admin (any employee); Manager (team member); any other role (own record)
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor requests an employee by EmployeeId. 2. System evaluates the actor's scope (All, Team or Own). 3. System returns the employee record.
Validation / Business Rules	BR-EMP-005, BR-GEN-002.
Expected Result	HTTP 200 with all Employee fields.
Error Conditions	• EmployeeId not found: 404 NOT_FOUND • Record outside the actor's scope: 403 FORBIDDEN • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/employees/{employeeId}

FR-EMP-003 — View All Employees	
Description	List all employees with paging and sorting.
Actor	HR, Admin
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor requests the employee list with optional pageNumber, pageSize and sort parameters. 2. System returns the requested page ordered by LastName then FirstName by default.
Validation / Business Rules	Default pageSize 20, maximum 100. Response includes totalCount. Deleted employees are not returned.
Expected Result	HTTP 200 with a page of employees and totalCount (empty page when none exist).
Error Conditions	• Invalid paging or sort parameter: 400 VALIDATION_ERROR • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/employees

FR-EMP-004 — Search Employees	
Description	Search and filter employees.
Actor	HR, Admin (all employees); Manager (team only)
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor submits a search text and/or filters DepartmentId, Role, EmploymentStatus. 2. System matches the text case-insensitively against FirstName, LastName and Email (partial match). 3. System combines all supplied criteria with AND, restricts results to the actor's scope, and returns a page of matches.
Validation / Business Rules	BR-EMP-005, BR-GEN-002. Paging rules as FR-EMP-003. Filter values for Role and EmploymentStatus must be valid enumeration values.
Expected Result	HTTP 200 with matching employees and totalCount (empty page when none match).
Error Conditions	• Invalid filter value: 400 VALIDATION_ERROR • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/employees/search

FR-EMP-005 — Update Employee	
Description	Modify an existing employee record.
Actor	HR, Admin
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor submits the changed fields for an EmployeeId. 2. System validates the values. 3. System verifies Email uniqueness excluding the employee being updated. 4. If DepartmentId changed, system verifies it exists through Department Service (REST). 5. System saves the changes and sets UpdatedAt.
Validation / Business Rules	BR-EMP-001, BR-EMP-003, BR-EMP-004, BR-EMP-007, BR-EMP-008, BR-EMP-009, BR-GEN-004.
Expected Result	HTTP 200 with the updated employee.
Error Conditions	• EmployeeId not found: 404 NOT_FOUND • Invalid field: 400 VALIDATION_ERROR • Duplicate Email: 409 CONFLICT • Unknown DepartmentId: 400 REFERENCE_NOT_FOUND • Department Service unavailable: 503 DEPENDENCY_UNAVAILABLE • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	PUT /api/v1/employees/{employeeId}

FR-EMP-006 — Delete Employee	
Description	Remove an employee record.
Actor	HR, Admin
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor requests deletion of an EmployeeId. 2. System verifies that the employee exists. 3. System deletes the employee so that it no longer appears in View or Search results.
Validation / Business Rules	BR-GEN-003: records in other services that reference the EmployeeId are not modified. The deletion mechanism is a design decision (Assumption A-06).
Expected Result	HTTP 204. The employee is no longer returned by FR-EMP-002, FR-EMP-003 or FR-EMP-004.
Error Conditions	• EmployeeId not found: 404 NOT_FOUND • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	DELETE /api/v1/employees/{employeeId}

FR-EMP-007 — View Own Profile	
Description	Retrieve the authenticated user's own employee record.
Actor	Any authenticated user
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor opens the Profile screen. 2. System resolves the EmployeeId from the access token. 3. System returns the employee record.
Validation / Business Rules	BR-GEN-006, BR-EMP-005.
Expected Result	HTTP 200 with EmployeeId, FirstName, LastName, Email, Phone, DateOfJoining, DepartmentId, Role and EmploymentStatus.
Error Conditions	• No employee record for the token's EmployeeId: 404 NOT_FOUND • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/employees/me

FR-EMP-008 — Update Own Profile	
Description	Allow a user to edit permitted fields of their own profile.
Actor	Any authenticated user
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor edits FirstName, LastName and/or Phone on the Profile screen and saves. 2. System resolves the EmployeeId from the access token. 3. System validates the values and rejects any non-editable field. 4. System saves the changes and sets UpdatedAt.
Validation / Business Rules	BR-EMP-003, BR-EMP-006, BR-GEN-004, BR-GEN-006. Email, DepartmentId, Role, EmploymentStatus and DateOfJoining are not editable by the employee.
Expected Result	HTTP 200 with the updated profile.
Error Conditions	• Invalid value: 400 VALIDATION_ERROR • Non-editable field supplied: 400 VALIDATION_ERROR • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	PUT /api/v1/employees/me

FR-EMP-009 — View Team Employees	
Description	List the employees who belong to the Manager's team.
Actor	Manager
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Manager requests the team list with optional paging parameters. 2. System determines the team members of the Manager (Assumption A-03). 3. System returns a page of team employee records.
Validation / Business Rules	BR-EMP-005, BR-GEN-002. Paging rules as FR-EMP-003.
Expected Result	HTTP 200 with team members and totalCount (empty page when the team is empty).
Error Conditions	• Invalid paging parameter: 400 VALIDATION_ERROR • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/employees/team

14.4 Department Service

Service base path: /api/v1/departments. Database: DepartmentDB.

FR-DEP-001 — Add Department	
Description	Create a department.
Actor	HR, Admin
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor submits DepartmentName and optional Description. 2. System validates the values. 3. System verifies DepartmentName is unique. 4. System generates DepartmentId, sets CreatedAt and UpdatedAt, and stores the record.
Validation / Business Rules	BR-DEP-001, BR-DEP-003, BR-GEN-004.
Expected Result	HTTP 201 with the new DepartmentId.
Error Conditions	• Missing or invalid field: 400 VALIDATION_ERROR • Duplicate DepartmentName: 409 CONFLICT • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	POST /api/v1/departments

FR-DEP-002 — View Department	
Description	Retrieve one department by DepartmentId.
Actor	Any authenticated user
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor requests a department by DepartmentId. 2. System returns the department record.
Validation / Business Rules	Department data is non-sensitive and readable by every authenticated role.
Expected Result	HTTP 200 with DepartmentId, DepartmentName, Description, CreatedAt and UpdatedAt.
Error Conditions	• DepartmentId not found: 404 NOT_FOUND • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/departments/{departmentId}

FR-DEP-003 — View All Departments	
Description	List all departments.
Actor	Any authenticated user
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor requests the department list with optional paging and search text. 2. System returns a page ordered by DepartmentName.
Validation / Business Rules	Paging rules as FR-EMP-003. Optional search text matches DepartmentName case-insensitively (partial match).
Expected Result	HTTP 200 with a page of departments and totalCount.
Error Conditions	• Invalid paging parameter: 400 VALIDATION_ERROR • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/departments

FR-DEP-004 — Update Department	
Description	Modify a department's name or description.
Actor	HR, Admin
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor submits changed DepartmentName and/or Description for a DepartmentId. 2. System validates the values. 3. System verifies DepartmentName uniqueness excluding the department being updated. 4. System saves the changes and sets UpdatedAt.
Validation / Business Rules	BR-DEP-001, BR-DEP-003, BR-GEN-004.
Expected Result	HTTP 200 with the updated department.
Error Conditions	• DepartmentId not found: 404 NOT_FOUND • Invalid field: 400 VALIDATION_ERROR • Duplicate DepartmentName: 409 CONFLICT • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	PUT /api/v1/departments/{departmentId}

FR-DEP-005 — Delete Department	
Description	Delete a department that has no assigned employees.
Actor	HR, Admin
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor requests deletion of a DepartmentId. 2. System verifies the department exists. 3. System asks Employee Service (REST) for the number of employees assigned to the DepartmentId. 4. If the number is zero, system deletes the department; otherwise it rejects the request.
Validation / Business Rules	BR-DEP-002. If Employee Service cannot be reached the department is not deleted.
Expected Result	HTTP 204 when deleted.
Error Conditions	• DepartmentId not found: 404 NOT_FOUND • Employees still assigned: 409 CONFLICT • Employee Service unavailable: 503 DEPENDENCY_UNAVAILABLE • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	DELETE /api/v1/departments/{departmentId}

14.5 Salary Service

Service base path: /api/v1/salaries. Database: SalaryDB. Employees can only view salary information and submit overtime requests; HR and Admin manage salary, components, revisions, bonuses and payroll.

Employee Salary

FR-SAL-001 — Add Salary	
Description	Create the current salary record of an employee.
Actor	HR, Admin
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor submits EmployeeId, BaseSalary and EffectiveFrom. 2. System validates the values. 3. System verifies through Employee Service (REST) that the EmployeeId exists. 4. System verifies that no salary record exists for the EmployeeId. 5. System generates SalaryId, stores the record and publishes SalaryUpdated (ChangeType Created).
Validation / Business Rules	BR-SAL-001, BR-SAL-002, BR-SAL-004, BR-SAL-007, BR-GEN-004.
Expected Result	HTTP 201 with the new SalaryId.
Error Conditions	• Invalid amount or date: 400 VALIDATION_ERROR • Unknown EmployeeId: 400 REFERENCE_NOT_FOUND • Salary already exists: 409 CONFLICT • Employee role attempts the operation: 403 FORBIDDEN • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	POST /api/v1/salaries

FR-SAL-002 — View Salary	
Description	View the current salary record.
Actor	HR, Admin (any employee); any other role (own salary)
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor requests own salary (me) or, for HR and Admin, a salary by SalaryId. 2. System authorises the request against BR-SAL-003. 3. System returns the salary record.
Validation / Business Rules	BR-SAL-003, BR-SAL-014. Managers have no access to team members' salary.
Expected Result	HTTP 200 with SalaryId, EmployeeId, BaseSalary, EffectiveFrom, CreatedAt and UpdatedAt.
Error Conditions	• No salary record: 404 NOT_FOUND • Another employee's salary requested: 403 FORBIDDEN • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/salaries/me GET /api/v1/salaries/{salaryId}

FR-SAL-003 — Update Salary	
Description	Correct the current salary record.
Actor	HR, Admin
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor submits corrected BaseSalary and/or EffectiveFrom for a SalaryId. 2. System validates the values. 3. System saves the changes, sets UpdatedAt and publishes SalaryUpdated (ChangeType Updated).
Validation / Business Rules	BR-SAL-001, BR-SAL-002, BR-SAL-007, BR-GEN-004. A formal change of pay is recorded through Salary Revision (FR-SAL-007).
Expected Result	HTTP 200 with the updated salary.
Error Conditions	• SalaryId not found: 404 NOT_FOUND • Invalid amount or date: 400 VALIDATION_ERROR • Employee role attempts the operation: 403 FORBIDDEN • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	PUT /api/v1/salaries/{salaryId}

Salary Components

FR-SAL-004 — Add Salary Component	
Description	Add a component to a salary.
Actor	HR, Admin
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor submits ComponentName, ComponentType (Earning or Deduction) and Amount for a SalaryId. 2. System validates the values. 3. System verifies ComponentName is unique within the salary. 4. System stores the component and publishes SalaryUpdated (ChangeType ComponentAdded).
Validation / Business Rules	BR-SAL-001, BR-SAL-002, BR-SAL-007.
Expected Result	HTTP 201 with the new ComponentId.
Error Conditions	• SalaryId not found: 404 NOT_FOUND • Invalid value: 400 VALIDATION_ERROR • Duplicate ComponentName: 409 CONFLICT • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	POST /api/v1/salaries/{salaryId}/components

FR-SAL-005 — Update Salary Component	
Description	Modify an existing salary component.
Actor	HR, Admin
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor submits changed ComponentName, ComponentType and/or Amount for a ComponentId. 2. System validates the values. 3. System saves the changes, sets UpdatedAt and publishes SalaryUpdated (ChangeType ComponentUpdated).
Validation / Business Rules	BR-SAL-001, BR-SAL-002, BR-SAL-007, BR-GEN-004.
Expected Result	HTTP 200 with the updated component.
Error Conditions	• Component not found: 404 NOT_FOUND • Invalid value: 400 VALIDATION_ERROR • Duplicate ComponentName: 409 CONFLICT • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	PUT /api/v1/salaries/{salaryId}/components/{componentId}

FR-SAL-006 — View Salary Components	
Description	List the components of a salary.
Actor	HR, Admin (any employee); any other role (own salary)
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor requests the components of own salary (me) or, for HR and Admin, of a SalaryId. 2. System authorises the request. 3. System returns the components.
Validation / Business Rules	BR-SAL-003, BR-SAL-014.
Expected Result	HTTP 200 with a list of ComponentId, ComponentName, ComponentType and Amount.
Error Conditions	• Salary not found: 404 NOT_FOUND • Another employee's salary: 403 FORBIDDEN • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/salaries/me/components GET /api/v1/salaries/{salaryId}/components

Salary Revision

FR-SAL-007 — Create Salary Revision	
Description	Record a salary revision and make it the current salary.
Actor	HR, Admin
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor submits SalaryId, NewAmount, EffectiveDate and Reason. 2. System validates the values. 3. System captures the current BaseSalary as PreviousAmount. 4. In one transaction, system stores the revision and sets BaseSalary to NewAmount. 5. System publishes SalaryUpdated (ChangeType Revised).
Validation / Business Rules	BR-SAL-001, BR-SAL-002, BR-SAL-005, BR-SAL-006, BR-SAL-007.
Expected Result	HTTP 201 with the new RevisionId.
Error Conditions	• SalaryId not found: 404 NOT_FOUND • NewAmount invalid or equal to current: 400 VALIDATION_ERROR • Employee role attempts the operation: 403 FORBIDDEN • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	POST /api/v1/salaries/{salaryId}/revisions

FR-SAL-008 — View Salary Revision History	
Description	List the salary revisions of an employee.
Actor	HR, Admin (any employee); any other role (own salary)
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor requests the revision history of own salary (me) or, for HR and Admin, of a SalaryId. 2. System authorises the request. 3. System returns revisions ordered by EffectiveDate, newest first.
Validation / Business Rules	BR-SAL-003, BR-SAL-005, BR-SAL-014.
Expected Result	HTTP 200 with RevisionId, PreviousAmount, NewAmount, EffectiveDate, Reason, CreatedBy and CreatedAt for each revision.
Error Conditions	• Salary not found: 404 NOT_FOUND • Another employee's salary: 403 FORBIDDEN • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/salaries/me/revisions GET /api/v1/salaries/{salaryId}/revisions

Bonus

FR-SAL-009 — Add Bonus	
Description	Record a bonus for an employee.
Actor	HR, Admin
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor submits EmployeeId, Amount, Reason, BonusDate and Status (Pending or Approved). 2. System validates the values. 3. System verifies through Employee Service (REST) that the EmployeeId exists. 4. System generates BonusId and stores the bonus.
Validation / Business Rules	BR-SAL-001, BR-SAL-002, BR-SAL-007, BR-SAL-013, BR-GEN-004.
Expected Result	HTTP 201 with the new BonusId.
Error Conditions	• Invalid value: 400 VALIDATION_ERROR • Unknown EmployeeId: 400 REFERENCE_NOT_FOUND • Employee role attempts the operation: 403 FORBIDDEN • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	POST /api/v1/salaries/bonuses

FR-SAL-010 — Update Bonus	
Description	Modify a bonus that is not yet part of a payroll.
Actor	HR, Admin
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor submits changed Amount, Reason, BonusDate and/or Status for a BonusId. 2. System verifies the bonus is not included in a payroll. 3. System validates and saves the changes and sets UpdatedAt.
Validation / Business Rules	BR-SAL-001, BR-SAL-002, BR-SAL-007, BR-SAL-013, BR-GEN-004.
Expected Result	HTTP 200 with the updated bonus.
Error Conditions	• BonusId not found: 404 NOT_FOUND • Invalid value: 400 VALIDATION_ERROR • Bonus already included in a payroll: 409 CONFLICT • Employee role attempts the operation: 403 FORBIDDEN • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	PUT /api/v1/salaries/bonuses/{bonusId}

FR-SAL-011 — View Bonus	
Description	List bonuses.
Actor	HR, Admin (all bonuses); any other role (own Approved bonuses)
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor requests bonuses with optional filters (EmployeeId and Status for HR and Admin). 2. System restricts the result to the actor's scope. 3. System returns a page of bonuses.
Validation / Business Rules	BR-SAL-003, BR-SAL-013, BR-SAL-014. Non-HR/Admin actors receive only their own bonuses with Status Approved.
Expected Result	HTTP 200 with bonuses and totalCount.
Error Conditions	• Invalid filter: 400 VALIDATION_ERROR • Another employee's EmployeeId requested by a non-HR/Admin actor: 403 FORBIDDEN • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/salaries/bonuses

Overtime

FR-SAL-012 — Submit Overtime Request	
Description	Submit an overtime request for the authenticated employee.
Actor	Any authenticated user (for own overtime)
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor submits OvertimeDate, Hours and Reason. 2. System resolves the EmployeeId from the access token. 3. System validates the values. 4. System stores the request with Status Pending.
Validation / Business Rules	BR-SAL-008, BR-GEN-004, BR-GEN-006.
Expected Result	HTTP 201 with the new OvertimeId and Status Pending.
Error Conditions	• Hours outside range or missing Reason: 400 VALIDATION_ERROR • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	POST /api/v1/salaries/overtime

FR-SAL-013 — View Overtime Requests	
Description	List overtime requests and approved overtime.
Actor	Employee and other roles (own); Manager (team); HR, Admin (all)
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor requests overtime with optional filters (Status, EmployeeId, date range). 2. System restricts the result to the actor's scope. 3. System returns a page of overtime requests.
Validation / Business Rules	BR-SAL-003, BR-GEN-002. Filtering by Status Approved returns the approved overtime an employee is entitled to view.
Expected Result	HTTP 200 with overtime requests and totalCount.
Error Conditions	• Invalid filter: 400 VALIDATION_ERROR • EmployeeId outside the actor's scope: 403 FORBIDDEN • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/salaries/overtime

FR-SAL-014 — Approve Overtime	
Description	Approve a pending overtime request.
Actor	Manager (team member's request), HR, Admin
Preconditions	The overtime request has Status Pending.
Main Flow	1. Actor selects a pending request and approves it. 2. System verifies scope and that the actor is not the requester. 3. System sets Status Approved, ReviewedBy and ReviewedAt. 4. System publishes OvertimeApproved.
Validation / Business Rules	BR-SAL-009, BR-GEN-005.
Expected Result	HTTP 200 with the updated request.
Error Conditions	• OvertimeId not found: 404 NOT_FOUND • Status is not Pending: 409 CONFLICT • Actor is the requester or outside scope: 403 FORBIDDEN • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	PUT /api/v1/salaries/overtime/{overtimeId}/approve

FR-SAL-015 — Reject Overtime	
Description	Reject a pending overtime request.
Actor	Manager (team member's request), HR, Admin
Preconditions	The overtime request has Status Pending.
Main Flow	1. Actor selects a pending request and rejects it, optionally with a comment (maximum 500 characters). 2. System verifies scope and that the actor is not the requester. 3. System sets Status Rejected, ReviewedBy and ReviewedAt.
Validation / Business Rules	BR-SAL-009, BR-GEN-005. No Kafka event is defined for rejection.
Expected Result	HTTP 200 with the updated request.
Error Conditions	• OvertimeId not found: 404 NOT_FOUND • Status is not Pending: 409 CONFLICT • Actor is the requester or outside scope: 403 FORBIDDEN • Comment too long: 400 VALIDATION_ERROR • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	PUT /api/v1/salaries/overtime/{overtimeId}/reject

FR-SAL-016 — Include Approved Overtime in Payroll	
Description	Add approved overtime to a generated payroll.
Actor	HR, Admin
Preconditions	The payroll has Status Generated and the overtime has Status Approved.
Main Flow	1. Actor selects a payroll and one or more approved overtime requests and supplies the overtime amount for each. 2. System verifies each request is Approved and not yet included. 3. System creates an Overtime payroll item for each request and marks the request as included. 4. System updates the employee's payroll totals.
Validation / Business Rules	BR-SAL-001, BR-SAL-007, BR-SAL-010. The overtime amount is supplied by the actor; automatic rate calculation is out of scope (Assumption A-08).
Expected Result	HTTP 200 with the created payroll items.
Error Conditions	• Payroll or overtime not found: 404 NOT_FOUND • Payroll not in Generated status, or overtime not Approved or already included: 409 CONFLICT • Invalid amount: 400 VALIDATION_ERROR • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	POST /api/v1/salaries/payrolls/{payrollId}/overtime

Payroll

FR-SAL-017 — Generate Payroll	
Description	Generate the payroll for a calendar month.
Actor	HR, Admin
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor submits PayrollMonth and PayrollYear. 2. System verifies that no payroll exists for that period. 3. System identifies employees who have a salary record and EmploymentStatus Active (verified through Employee Service). 4. For each such employee system creates payroll items: one BasicSalary item, one item per salary component, and one Bonus item per Approved bonus with BonusDate in the period not yet included. 5. System stores the payroll with Status Generated.
Validation / Business Rules	BR-SAL-010, BR-SAL-011, BR-SAL-013.
Expected Result	HTTP 201 with PayrollId, period, Status Generated and the number of employees included.
Error Conditions	• Invalid month or year: 400 VALIDATION_ERROR • Payroll already exists for the period: 409 CONFLICT • No eligible employees: 409 CONFLICT • Employee Service unavailable: 503 DEPENDENCY_UNAVAILABLE • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	POST /api/v1/salaries/payrolls

FR-SAL-018 — View Payroll	
Description	View payroll runs.
Actor	HR, Admin
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor requests the payroll list (optionally by year) or one payroll by PayrollId. 2. System returns the payroll header data.
Validation / Business Rules	BR-SAL-014.
Expected Result	HTTP 200 with PayrollId, PayrollMonth, PayrollYear, Status, GeneratedBy, GeneratedAt and employee count.
Error Conditions	• PayrollId not found: 404 NOT_FOUND • Invalid filter: 400 VALIDATION_ERROR • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/salaries/payrolls GET /api/v1/salaries/payrolls/{payrollId}

FR-SAL-019 — View Payroll Items	
Description	View the items of a payroll.
Actor	HR, Admin
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor requests the items of a PayrollId with optional EmployeeId filter. 2. System returns a page of payroll items.
Validation / Business Rules	BR-SAL-014.
Expected Result	HTTP 200 with PayrollItemId, EmployeeId, ItemType, Description and Amount for each item.
Error Conditions	• PayrollId not found: 404 NOT_FOUND • Invalid filter: 400 VALIDATION_ERROR • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/salaries/payrolls/{payrollId}/items

Payslip

FR-SAL-020 — Generate Payslip	
Description	Generate payslips for all employees in a payroll.
Actor	HR, Admin
Preconditions	The payroll has Status Generated.
Main Flow	1. Actor requests payslip generation for a PayrollId. 2. System calculates TotalEarnings, TotalDeductions and NetPay for each employee from the payroll items. 3. System creates one payslip per employee. 4. System sets the payroll Status to Finalized.
Validation / Business Rules	BR-SAL-012, BR-SAL-014.
Expected Result	HTTP 201 with the number of payslips generated; payroll is Finalized.
Error Conditions	• PayrollId not found: 404 NOT_FOUND • Payroll not in Generated status: 409 CONFLICT • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	POST /api/v1/salaries/payrolls/{payrollId}/payslips

FR-SAL-021 — View Payslip	
Description	View payslips.
Actor	HR, Admin (any employee); any other role (own payslips)
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor requests a payslip list (optionally by PayrollMonth and PayrollYear) or one payslip by PayslipId. 2. System restricts the result to the actor's scope. 3. System returns the payslip data including its payroll item lines.
Validation / Business Rules	BR-SAL-003, BR-SAL-014.
Expected Result	HTTP 200 with PayslipId, EmployeeId, period, TotalEarnings, TotalDeductions, NetPay and item lines.
Error Conditions	• PayslipId not found: 404 NOT_FOUND • Another employee's payslip: 403 FORBIDDEN • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/salaries/payslips GET /api/v1/salaries/payslips/{payslipId}

14.6 Leave Service

Service base path: /api/v1/leaves. Database: LeaveDB. Workflow: Employee > Apply Leave > Pending > Approved or Rejected.

FR-LEV-001 — Apply Leave	
Description	Submit a leave request for the authenticated employee.
Actor	Any authenticated user (for own leave)
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor submits LeaveType, StartDate, EndDate and Reason. 2. System resolves the EmployeeId from the access token. 3. System validates the values. 4. System generates Leaveld, sets Status Pending and AppliedAt, and stores the request.
Validation / Business Rules	BR-LEV-001, BR-LEV-002, BR-LEV-004, BR-LEV-006, BR-GEN-006.
Expected Result	HTTP 201 with Leaveld and Status Pending.
Error Conditions	• StartDate later than EndDate: 400 VALIDATION_ERROR • Missing Reason or invalid LeaveType: 400 VALIDATION_ERROR • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	POST /api/v1/leaves

FR-LEV-002 — View Leave	
Description	View a single leave request and its status.
Actor	Employee and other roles (own); Manager (team); HR, Admin (all)
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor requests a leave by Leaveld. 2. System authorises the request against the actor's scope. 3. System returns the leave record.
Validation / Business Rules	BR-LEV-004, BR-GEN-002. Admin has read-only access.
Expected Result	HTTP 200 with all Leave fields including Status, ApprovedBy and ApprovedAt.
Error Conditions	• Leaveld not found: 404 NOT_FOUND • Outside the actor's scope: 403 FORBIDDEN • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/leaves/{leaveld}

FR-LEV-003 — View Leave History	
Description	List the authenticated user's own leave requests.
Actor	Any authenticated user (own leave)
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor requests own leave history with optional filters (Status, date range) and paging. 2. System returns a page ordered by AppliedAt, newest first.
Validation / Business Rules	BR-LEV-004, BR-GEN-006.
Expected Result	HTTP 200 with the leave records and totalCount.
Error Conditions	• Invalid filter or date range: 400 VALIDATION_ERROR • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/leaves/history

FR-LEV-004 — View Leave Requests	
Description	List leave requests for review.
Actor	Manager (team); HR, Admin (all)
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor requests leave requests with optional filters (Status, EmployeeId, date range); Status defaults to Pending. 2. System restricts the result to the actor's scope. 3. System returns a page ordered by AppliedAt.
Validation / Business Rules	BR-GEN-002. Admin has read-only access.
Expected Result	HTTP 200 with the leave requests and totalCount.
Error Conditions	• Invalid filter: 400 VALIDATION_ERROR • EmployeeId outside the actor's scope: 403 FORBIDDEN • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/leaves/requests

FR-LEV-005 — Approve Leave	
Description	Approve a pending leave request.
Actor	Manager (team member's request), HR
Preconditions	The leave has Status Pending.
Main Flow	1. Actor selects a pending leave and approves it. 2. System verifies scope and that the actor is not the applicant. 3. System sets Status Approved, ApprovedBy and ApprovedAt. 4. System publishes LeaveApproved.
Validation / Business Rules	BR-LEV-003, BR-LEV-005, BR-GEN-005.
Expected Result	HTTP 200 with the updated leave.
Error Conditions	• LeaveId not found: 404 NOT_FOUND • Status is not Pending: 409 CONFLICT • Actor is the applicant or outside scope: 403 FORBIDDEN • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	PUT /api/v1/leaves/{leaveId}/approve

FR-LEV-006 — Reject Leave	
Description	Reject a pending leave request.
Actor	Manager (team member's request), HR
Preconditions	The leave has Status Pending.
Main Flow	1. Actor selects a pending leave and rejects it. 2. System verifies scope and that the actor is not the applicant. 3. System sets Status Rejected, ApprovedBy and ApprovedAt. 4. System publishes LeaveRejected.
Validation / Business Rules	BR-LEV-003, BR-LEV-005, BR-GEN-005.
Expected Result	HTTP 200 with the updated leave.
Error Conditions	• LeaveId not found: 404 NOT_FOUND • Status is not Pending: 409 CONFLICT • Actor is the applicant or outside scope: 403 FORBIDDEN • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	PUT /api/v1/leaves/{leaveId}/reject

14.7 Attendance Service

Service base path: /api/v1/attendance. Database: AttendanceDB.

FR-ATT-001 — Check In	
Description	Record the start of the authenticated employee's working day.
Actor	Any authenticated user (own attendance)
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor selects Check In. 2. System resolves the EmployeeId from the token and takes the current server date and time. 3. System verifies that no record exists for the EmployeeId and date. 4. System creates the record with CheckInTime and Status CheckedIn.
Validation / Business Rules	BR-ATT-001, BR-ATT-003, BR-ATT-004, BR-GEN-006.
Expected Result	HTTP 201 with AttendanceId, AttendanceDate and CheckInTime.
Error Conditions	• Already checked in for the date: 409 CONFLICT • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	POST /api/v1/attendance/check-in

FR-ATT-002 — Check Out	
Description	Record the end of the authenticated employee's working day.
Actor	Any authenticated user (own attendance)
Preconditions	The employee has checked in for the current date.
Main Flow	1. Actor selects Check Out. 2. System locates the record for the EmployeeId and current date. 3. System verifies that CheckOutTime is not earlier than CheckInTime. 4. System sets CheckOutTime and Status CheckedOut.
Validation / Business Rules	BR-ATT-002, BR-ATT-003, BR-ATT-004, BR-GEN-004.
Expected Result	HTTP 200 with the completed attendance record.
Error Conditions	• No check-in for the date: 409 CONFLICT • Already checked out: 409 CONFLICT • CheckOutTime earlier than CheckInTime: 400 VALIDATION_ERROR • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	POST /api/v1/attendance/check-out

FR-ATT-003 — View Attendance	
Description	View the authenticated employee's attendance record for a date.
Actor	Any authenticated user (own attendance)
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor requests own attendance for a date (default: today). 2. System returns the record.
Validation / Business Rules	BR-ATT-005, BR-GEN-006.
Expected Result	HTTP 200 with the attendance record, or 404 when none exists for the date.
Error Conditions	• Invalid date: 400 VALIDATION_ERROR • No record for the date: 404 NOT_FOUND • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/attendance/me

FR-ATT-004 — View Attendance History	
Description	List the authenticated employee's attendance over a period.
Actor	Any authenticated user (own attendance)
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor requests own attendance with a from and to date (default: last 30 days) and paging. 2. System returns a page ordered by AttendanceDate, newest first.
Validation / Business Rules	BR-ATT-005. Maximum range 366 days.
Expected Result	HTTP 200 with attendance records and totalCount.
Error Conditions	• Invalid dates, from later than to, or range above the maximum: 400 VALIDATION_ERROR • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/attendance/history

FR-ATT-005 — View Team Attendance	
Description	View attendance of employees within the actor's authority.
Actor	Manager (team); HR, Admin (all employees)
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor requests attendance with a date or date range and an optional EmployeeId. 2. System determines the employees in scope through Employee Service (REST). 3. System returns a page of attendance records for the employees in scope.
Validation / Business Rules	BR-ATT-005, BR-GEN-002. Maximum range 366 days.
Expected Result	HTTP 200 with attendance records and totalCount.
Error Conditions	• Invalid dates: 400 VALIDATION_ERROR • EmployeeId outside the actor's scope: 403 FORBIDDEN • Employee Service unavailable: 503 DEPENDENCY_UNAVAILABLE • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/attendance/team

14.8 Support Service

Service base path: /api/v1/support-tickets. Database: SupportDB. Workflow: Open > Assigned > InProgress > Resolved > Closed.

FR-SUP-001 — Create Support Ticket	
Description	Raise a support ticket.
Actor	Any authenticated user (own ticket)
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor submits Title, Description and Priority. 2. System resolves the EmployeeId from the token and validates the values. 3. System generates TicketId and stores the ticket with Status Open, AssignedTo empty and CreatedAt set.
Validation / Business Rules	BR-SUP-001, BR-SUP-002, BR-GEN-006.
Expected Result	HTTP 201 with TicketId and Status Open.
Error Conditions	• Missing Title or Description, or invalid Priority: 400 VALIDATION_ERROR • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	POST /api/v1/support-tickets

FR-SUP-002 — View Ticket	
Description	View the details of one ticket.
Actor	Employee and other roles (own tickets); Support Staff, Admin (any ticket)
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor requests a ticket by TicketId. 2. System authorises the request. 3. System returns the ticket.
Validation / Business Rules	BR-SUP-004, BR-SUP-005.
Expected Result	HTTP 200 with all Ticket fields.
Error Conditions	• TicketId not found: 404 NOT_FOUND • Another employee's ticket: 403 FORBIDDEN • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/support-tickets/{ticketId}

FR-SUP-003 — View Tickets	
Description	List tickets.
Actor	Employee and other roles (own tickets); Support Staff, Admin (all tickets)
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor requests tickets with optional filters (Status, Priority, AssignedTo) and paging. 2. System restricts the result to the actor's scope. 3. System returns a page ordered by CreatedAt, newest first.
Validation / Business Rules	BR-SUP-004, BR-SUP-005.
Expected Result	HTTP 200 with tickets and totalCount.
Error Conditions	• Invalid filter value: 400 VALIDATION_ERROR • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/support-tickets

FR-SUP-004 — Assign Ticket	
Description	Assign a ticket to a Support Staff member.
Actor	Support Staff, Admin
Preconditions	The ticket has Status Open or Assigned.
Main Flow	1. Actor selects a ticket and the AssignedTo employee. 2. System verifies through Employee Service (REST) that AssignedTo exists and has Role SupportStaff. 3. System sets AssignedTo, Status Assigned and UpdatedAt. 4. System publishes SupportTicketUpdated.
Validation / Business Rules	BR-SUP-003, BR-SUP-005, BR-SUP-006.
Expected Result	HTTP 200 with the updated ticket.
Error Conditions	• TicketId not found: 404 NOT_FOUND • Ticket status not Open or Assigned: 409 CONFLICT • AssignedTo not found or not SupportStaff: 400 REFERENCE_NOT_FOUND • Employee Service unavailable: 503 DEPENDENCY_UNAVAILABLE • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	PUT /api/v1/support-tickets/{ticketId}/assign

FR-SUP-005 — Update Ticket Status	
Description	Move an assigned ticket to InProgress.
Actor	Support Staff (assignee), Admin
Preconditions	The ticket has Status Assigned.
Main Flow	1. Assignee selects Status InProgress. 2. System verifies the actor is the assignee (or Admin) and the transition is allowed. 3. System sets Status InProgress and UpdatedAt. 4. System publishes SupportTicketUpdated.
Validation / Business Rules	BR-SUP-003, BR-SUP-005.
Expected Result	HTTP 200 with the updated ticket.
Error Conditions	• Ticket not found: 404 NOT_FOUND • Invalid transition: 409 CONFLICT • Actor is not the assignee: 403 FORBIDDEN • Status value other than InProgress: 400 VALIDATION_ERROR • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	PUT /api/v1/support-tickets/{ticketId}/status

FR-SUP-006 — Resolve Ticket	
Description	Mark an in-progress ticket as resolved.
Actor	Support Staff (assignee), Admin
Preconditions	The ticket has Status InProgress.
Main Flow	1. Assignee selects Resolve. 2. System verifies the actor is the assignee (or Admin) and the transition is allowed. 3. System sets Status Resolved, ResolvedAt and UpdatedAt. 4. System publishes SupportTicketUpdated.
Validation / Business Rules	BR-SUP-003, BR-SUP-005, BR-SUP-007.
Expected Result	HTTP 200 with the updated ticket including ResolvedAt.
Error Conditions	• Ticket not found: 404 NOT_FOUND • Invalid transition: 409 CONFLICT • Actor is not the assignee: 403 FORBIDDEN • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	PUT /api/v1/support-tickets/{ticketId}/resolve

FR-SUP-007 — Close Ticket	
Description	Close a resolved ticket.
Actor	Support Staff (assignee), Admin
Preconditions	The ticket has Status Resolved.
Main Flow	1. Assignee selects Close. 2. System verifies the actor is the assignee (or Admin) and the transition is allowed. 3. System sets Status Closed and UpdatedAt. 4. System publishes SupportTicketUpdated.
Validation / Business Rules	BR-SUP-003, BR-SUP-005.
Expected Result	HTTP 200 with the updated ticket.
Error Conditions	• Ticket not found: 404 NOT_FOUND • Invalid transition: 409 CONFLICT • Actor is not the assignee: 403 FORBIDDEN • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	PUT /api/v1/support-tickets/{ticketId}/close

14.9 Notification Service

Service base path: /api/v1/notifications. Database: NotificationDB.

FR-NOT-001 — Create Notification from Event	
Description	Consume business events from Kafka and create notifications.
Actor	Notification Service (system)
Preconditions	The event is published to its Kafka topic.
Main Flow	1. Service consumes an event from a subscribed topic. 2. Service validates the event envelope and payload. 3. Service checks that the EventId has not been processed before. 4. Service maps the event to a notification (EmployeeId, Title, Message, Type) as defined in Section 21. 5. Service stores the notification with IsRead false and records the EventId.
Validation / Business Rules	BR-NOT-001, BR-NOT-002, BR-NOT-003. Handled event types: LeaveApproved, LeaveRejected, SalaryUpdated, OvertimeApproved, SupportTicketUpdated.
Expected Result	One unread notification exists for each recipient EmployeeId of the event.
Error Conditions	• Malformed event: routed to the dead-letter topic and logged • Duplicate EventId: ignored without creating a notification • Database failure: retried, then routed to the dead-letter topic
API Endpoint(s)	Kafka consumer (no REST endpoint)

FR-NOT-002 — View Notifications	
Description	List the authenticated user's notifications.
Actor	Any authenticated user (own notifications)
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor requests own notifications with optional filter isRead and paging. 2. System returns a page ordered by CreatedAt, newest first.
Validation / Business Rules	BR-NOT-001, BR-GEN-006.
Expected Result	HTTP 200 with NotificationId, Title, Message, Type, IsRead and CreatedAt, and totalCount.
Error Conditions	• Invalid filter or paging parameter: 400 VALIDATION_ERROR • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	GET /api/v1/notifications

FR-NOT-003 — Mark Notification as Read	
Description	Mark one notification as read.
Actor	Any authenticated user (own notifications)
Preconditions	Requester holds a valid access token and a role permitted for this operation.
Main Flow	1. Actor selects Mark as Read on a notification. 2. System verifies the notification belongs to the actor. 3. System sets IsRead to true.
Validation / Business Rules	BR-NOT-001, BR-NOT-004. Repeating the request has no further effect.
Expected Result	HTTP 200 with the updated notification.
Error Conditions	• NotificationId not found: 404 NOT_FOUND • Notification belongs to another user: 403 FORBIDDEN • 401 UNAUTHENTICATED (no or invalid token); 403 FORBIDDEN (role or scope not permitted) — see Section 14 conventions
API Endpoint(s)	PUT /api/v1/notifications/{notificationId}/read

15. User Workflows

15.1 Use Case Summary

ID	Use case	Primary actor(s)	Related requirements
UC-01	Log in	All users	FR-AUT-001
UC-02	Recover a forgotten password	All users	FR-AUT-002, FR-AUT-003
UC-03	Manage user accounts and roles	Admin	FR-AUT-004, FR-AUT-005
UC-04	Manage employee records	HR, Admin	FR-EMP-001, FR-EMP-005, FR-EMP-006
UC-05	View, list and search employees	HR, Admin, Manager (team)	FR-EMP-002, FR-EMP-003, FR-EMP-004, FR-EMP-009
UC-06	Maintain own profile	All users	FR-EMP-007, FR-EMP-008
UC-07	Manage departments	HR, Admin	FR-DEP-001, FR-DEP-004, FR-DEP-005
UC-08	View departments	All users	FR-DEP-002, FR-DEP-003
UC-09	Manage salary and salary components	HR, Admin	FR-SAL-001 to FR-SAL-006
UC-10	Revise salary and view revision history	HR, Admin	FR-SAL-007, FR-SAL-008
UC-11	Manage bonuses	HR, Admin	FR-SAL-009, FR-SAL-010, FR-SAL-011
UC-12	View own salary information	Employee	FR-SAL-002, FR-SAL-006, FR-SAL-008, FR-SAL-011
UC-13	Submit and track overtime	Employee	FR-SAL-012, FR-SAL-013
UC-14	Review overtime	Manager (team), HR, Admin	FR-SAL-013, FR-SAL-014, FR-SAL-015
UC-15	Prepare payroll	HR, Admin	FR-SAL-016 to FR-SAL-019
UC-16	Generate and view payslips	HR, Admin (generate); Employee (own)	FR-SAL-020, FR-SAL-021
UC-17	Apply for leave and track status	Employee	FR-LEV-001, FR-LEV-002, FR-LEV-003
UC-18	Review leave requests	Manager (team), HR	FR-LEV-004, FR-LEV-005, FR-LEV-006
UC-19	Record and view own attendance	Employee	FR-ATT-001 to FR-ATT-004
UC-20	View team attendance	Manager (team), HR, Admin	FR-ATT-005
UC-21	Raise and track a support ticket	Employee	FR-SUP-001, FR-SUP-002, FR-SUP-003
UC-22	Handle support tickets	Support Staff, Admin	FR-SUP-002 to FR-SUP-007
UC-23	Receive and read notifications	All users	FR-NOT-001, FR-NOT-002, FR-NOT-003

15.2 Major Workflows

WF-01 Leave Request

Employee > Apply Leave > Pending > Approved / Rejected (FR-LEV-001, FR-LEV-005, FR-LEV-006)

- Employee submits LeaveType, StartDate, EndDate and Reason. The request is stored with Status Pending.
- The applicant's Manager or HR views the pending request (FR-LEV-004) and approves or rejects it. The applicant cannot decide their own request.
- Leave Service sets Status, ApprovedBy and ApprovedAt and publishes LeaveApproved or LeaveRejected to Kafka.
- Notification Service creates a notification for the employee, who can also see the status through View Leave.

WF-02 Overtime, Payroll and Payslip

- Employee submits an overtime request (Pending) — FR-SAL-012.
- Manager (team), HR or Admin approves or rejects it — FR-SAL-014 / FR-SAL-015. Approval publishes OvertimeApproved and the employee is notified.
- HR or Admin generates the payroll for a month (Status Generated) — FR-SAL-017.

- HR or Admin includes approved overtime in the payroll — FR-SAL-016.
- HR or Admin generates payslips, which finalizes the payroll — FR-SAL-020.
- Each employee views their own payslip — FR-SAL-021.

WF-03 Salary Setup and Revision

- HR or Admin adds the salary record and its components (FR-SAL-001, FR-SAL-004).
- For a change in pay, HR or Admin creates a salary revision (FR-SAL-007). The revision is stored permanently and becomes the current salary in the same transaction.
- Salary Service publishes SalaryUpdated; the employee is notified and can view the salary and revision history. The employee cannot change any of this data.

WF-04 Support Ticket

Open > Assigned > InProgress > Resolved > Closed (FR-SUP-001 to FR-SUP-007)

- Employee creates a ticket (Open).
- Support Staff or Admin assigns it to a Support Staff member (Assigned).
- The assignee moves it to InProgress, then Resolves it (ResolvedAt set) and finally Closes it.
- Every assignment and status change publishes SupportTicketUpdated; the ticket owner is notified.

WF-05 Daily Attendance

- Employee checks in (record created, CheckedIn) and later checks out (CheckedOut). One record per employee per day.
- Managers view their team's attendance; HR and Admin view attendance of any employee.

WF-06 Employee and Department Maintenance

- HR or Admin creates departments, then employees; Employee Service checks each DepartmentId with Department Service.
- To delete a department, Department Service asks Employee Service whether employees are still assigned; deletion is refused if any are.

WF-07 Password Recovery

- User requests a reset link with the registered email (FR-AUT-002), receives a generic confirmation, then sets a new password with the token (FR-AUT-003).

16. Business Rules

The table below is the Business Rules Summary. Each rule is referenced by the functional requirements that enforce it.

16.1 General

Rule ID	Business rule
BR-GEN-001	Each user account is linked to exactly one EmployeeId. Every authenticated user, whatever the role, may perform the self-service operations on their own data that are defined for the Employee role.
BR-GEN-002	Access requires a role that permits the operation and, for employee-scoped data, satisfaction of the scope (All, Team, Own, Assigned) shown in the Role Permission Matrix.
BR-GEN-003	EmployeeId and DepartmentId held in another service are references validated through REST at write time. No physical foreign key exists across service databases, and deleting a record does not modify records that reference it in other services.
BR-GEN-004	CreatedAt and UpdatedAt are set by the server in UTC and cannot be supplied by clients. UpdatedAt changes on every modification.
BR-GEN-005	A user cannot approve, reject or review a request that they submitted themselves.
BR-GEN-006	For self-service operations the acting EmployeeId is taken from the access token and never from client input.

16.2 Employee Service

Rule ID	Business rule
BR-EMP-001	Employee Email is unique across all employees (case-insensitive).
BR-EMP-002	Required at creation: FirstName, LastName, Email, Phone, DateOfJoining, DepartmentId, Role, EmploymentStatus.
BR-EMP-003	Email must be a valid address (maximum 254 characters). Phone contains 7 to 15 digits with an optional leading +. FirstName and LastName have a maximum of 100 characters.
BR-EMP-004	DepartmentId must reference an existing department when an employee is created and whenever DepartmentId is changed.
BR-EMP-005	An employee's record may be read by that employee, by HR and Admin (all employees), and by a Manager for team members only. No other access is allowed, so private data of one employee is never available to an unauthorized user.
BR-EMP-006	An employee may edit only FirstName, LastName and Phone of their own profile.
BR-EMP-007	Role must be one of Admin, HR, Manager, Employee, SupportStaff.
BR-EMP-008	EmploymentStatus must be one of Active, Inactive.
BR-EMP-009	EmployeeId and CreatedAt are immutable.

16.3 Department Service

Rule ID	Business rule
BR-DEP-001	DepartmentName is required, has a maximum of 100 characters and is unique (case-insensitive).
BR-DEP-002	A department cannot be deleted while one or more employees are assigned to it.
BR-DEP-003	Description is optional with a maximum of 500 characters.

16.4 Salary Service

Rule ID	Business rule
BR-SAL-001	Employees cannot create, modify or delete salary, salary components, salary revisions or bonuses.
BR-SAL-002	Only HR and Admin can add or update salary, salary components and bonuses, and create salary revisions.
BR-SAL-003	An employee-level actor can see only their own salary, components, revision history, Approved bonuses, overtime requests and payslips. Managers have no access to team members' salary data.
BR-SAL-004	At most one current salary record exists per EmployeeId.
BR-SAL-005	Salary revisions are immutable. They cannot be edited or deleted, so revision history is complete.
BR-SAL-006	A revision records the previous amount automatically. NewAmount must be greater than zero and different from the current BaseSalary.
BR-SAL-007	Monetary amounts are non-negative decimals with at most two decimal places; BaseSalary must be greater than zero. ComponentName is unique within a salary.
BR-SAL-008	An overtime request is submitted by an employee for themselves, starts with Status Pending, has Hours greater than 0 and at most 24, and requires a Reason.
BR-SAL-009	Only Pending overtime can be approved or rejected, by a Manager (for team members), HR or Admin. BR-GEN-005 applies. A decision is final.
BR-SAL-010	Only HR and Admin can include overtime in payroll. Only Approved overtime not yet included can be included, only in a payroll with Status Generated, and each overtime request can be included at most once.
BR-SAL-011	At most one payroll exists per PayrollMonth and PayrollYear.
BR-SAL-012	Payslips are generated only for a payroll with Status Generated. There is one payslip per employee per payroll. After generation the payroll is Finalized and cannot be changed.
BR-SAL-013	Employees see only bonuses with Status Approved. An Approved bonus is included once in the payroll of the period containing its BonusDate. A bonus included in a payroll cannot be updated.
BR-SAL-014	Salary, bonus, payroll and payslip amounts are sensitive: they are returned only as permitted by BR-SAL-003, are never written to logs, and are never carried in Kafka events.

16.5 Leave Service

Rule ID	Business rule
BR-LEV-001	StartDate must be less than or equal to EndDate.
BR-LEV-002	A new leave request has Status Pending; AppliedAt is set by the server.
BR-LEV-003	Allowed transitions are Pending to Approved and Pending to Rejected only. Decisions are final.
BR-LEV-004	An employee applies for leave only for themselves and can view only their own leave requests and their status.
BR-LEV-005	Leave is approved or rejected by the applicant's Manager (team) or by HR. BR-GEN-005 applies. ApprovedBy and ApprovedAt record the decision maker and time for both approval and rejection.
BR-LEV-006	LeaveType is required and must be one of the configured leave types. Reason is required with a maximum of 500 characters.

16.6 Attendance Service

Rule ID	Business rule
BR-ATT-001	There is at most one attendance record per employee per day. The combination EmployeeId + AttendanceDate is unique.
BR-ATT-002	CheckOutTime cannot be earlier than CheckInTime.
BR-ATT-003	An employee can check in once per day. Check-out requires a prior check-in and can occur once per day.
BR-ATT-004	AttendanceDate, CheckInTime and CheckOutTime come from the server clock (UTC) and are not supplied by the client.
BR-ATT-005	Attendance is visible to the employee (own), to a Manager (team) and to HR and Admin (all).

16.7 Support Service

Rule ID	Business rule
BR-SUP-001	Title (maximum 150 characters) and Description (maximum 2000 characters) are required.
BR-SUP-002	Priority is required and must be one of Low, Medium, High.
BR-SUP-003	Status follows Open, Assigned, InProgress, Resolved, Closed in that order. Steps cannot be skipped or reversed. A ticket may be re-assigned while it is Assigned.
BR-SUP-004	Employees can access only their own tickets.
BR-SUP-005	Support Staff can view all tickets and assign them, but change status only on tickets assigned to them. Admin has full access.
BR-SUP-006	AssignedTo must identify an existing employee whose Role is SupportStaff.
BR-SUP-007	ResolvedAt is set when the ticket becomes Resolved and is empty before that.

16.8 Notification Service

Rule ID	Business rule
BR-NOT-001	A notification is visible only to the employee identified by its EmployeeId.
BR-NOT-002	Notifications are created only from consumed Kafka events and cannot be created through the REST API.
BR-NOT-003	Event processing is idempotent: an EventId creates notifications only once.
BR-NOT-004	IsRead is false on creation. Only the owner can mark a notification as read; marking is idempotent and cannot be reversed.

17. Data Requirements

Identifiers are UUIDs. The tables below give the logical data model of each service. Fields marked "Service reference" hold an identifier owned by another service and are never physical foreign keys. Entities and fields of the Salary Service other than those named in the requirements statement are derived from the required operations (Assumption A-07).

Employee (EmployeeDB.Employees)

Field	Type (PostgreSQL)	Required	Constraints / notes
EmployeeId	uuid	Yes	Primary key; system-generated
FirstName	varchar(100)	Yes	
LastName	varchar(100)	Yes	
Email	varchar(254)	Yes	Unique; valid e-mail format
Phone	varchar(20)	Yes	7 to 15 digits, optional leading +
DateOfJoining	date	Yes	Valid date
DepartmentId	uuid	Yes	Service reference to Department Service (no foreign key)
Role	varchar(20)	Yes	Admin, HR, Manager, Employee, SupportStaff
EmploymentStatus	varchar(20)	Yes	Active, Inactive
CreatedAt / UpdatedAt	timestampz	Yes	Server-set (UTC)

Department (DepartmentDB.Departments)

Field	Type (PostgreSQL)	Required	Constraints / notes
DepartmentId	uuid	Yes	Primary key; system-generated
DepartmentName	varchar(100)	Yes	Unique (case-insensitive)
Description	varchar(500)	No	
CreatedAt / UpdatedAt	timestampz	Yes	Server-set (UTC)

EmployeeSalary (SalaryDB.EmployeeSalaries)

Field	Type (PostgreSQL)	Required	Constraints / notes
SalaryId	uuid	Yes	Primary key
EmployeeId	uuid	Yes	Service reference; unique (one current salary per employee)
BaseSalary	numeric(12,2)	Yes	Greater than 0
EffectiveFrom	date	Yes	
CreatedAt / UpdatedAt	timestampz	Yes	Server-set (UTC)

SalaryComponent (SalaryDB.SalaryComponents)

Field	Type (PostgreSQL)	Required	Constraints / notes
ComponentId	uuid	Yes	Primary key
SalaryId	uuid	Yes	Foreign key to EmployeeSalaries (same database)
ComponentName	varchar(100)	Yes	Unique per SalaryId
ComponentType	varchar(10)	Yes	Earning, Deduction
Amount	numeric(12,2)	Yes	0 or greater
CreatedAt / UpdatedAt	timestampz	Yes	Server-set (UTC)

SalaryRevision (SalaryDB.SalaryRevisions)

Field	Type (PostgreSQL)	Required	Constraints / notes
RevisionId	uuid	Yes	Primary key

Field	Type (PostgreSQL)	Required	Constraints / notes
SalaryId	uuid	Yes	Foreign key to EmployeeSalaries (same database)
PreviousAmount / NewAmount	numeric(12,2)	Yes	NewAmount greater than 0 and different from PreviousAmount
EffectiveDate	date	Yes	
Reason	varchar(500)	Yes	
CreatedBy	uuid	Yes	Service reference (EmployeeId of HR/Admin)
CreatedAt	timestampz	Yes	Server-set; record is immutable

Bonus (SalaryDB.Bonuses)

Field	Type (PostgreSQL)	Required	Constraints / notes
BonusId	uuid	Yes	Primary key
EmployeeId	uuid	Yes	Service reference
Amount	numeric(12,2)	Yes	Greater than 0
Reason	varchar(500)	Yes	
BonusDate	date	Yes	
Status	varchar(10)	Yes	Pending, Approved
CreatedBy	uuid	Yes	Service reference
CreatedAt / UpdatedAt	timestampz	Yes	Server-set (UTC)

OvertimeRequest (SalaryDB.OvertimeRequests)

Field	Type (PostgreSQL)	Required	Constraints / notes
OvertimeId	uuid	Yes	Primary key
EmployeeId	uuid	Yes	Service reference
OvertimeDate	date	Yes	
Hours	numeric(4,2)	Yes	Greater than 0 and at most 24
Reason	varchar(500)	Yes	
Status	varchar(10)	Yes	Pending, Approved, Rejected
ReviewedBy / ReviewedAt	uuid / timestampz	No	Set on decision; ReviewedBy is a service reference
ReviewComment	varchar(500)	No	
IsIncludedInPayroll	boolean	Yes	Default false
CreatedAt / UpdatedAt	timestampz	Yes	Server-set (UTC)

Payroll (SalaryDB.Payrolls)

Field	Type (PostgreSQL)	Required	Constraints / notes
PayrollId	uuid	Yes	Primary key
PayrollMonth / PayrollYear	smallint / smallint	Yes	Month 1 to 12; unique together
Status	varchar(10)	Yes	Generated, Finalized
GeneratedBy / GeneratedAt	uuid / timestampz	Yes	GeneratedBy is a service reference

PayrollItem (SalaryDB.PayrollItems)

Field	Type (PostgreSQL)	Required	Constraints / notes
PayrollItemId	uuid	Yes	Primary key
PayrollId	uuid	Yes	Foreign key to Payrolls (same database)
EmployeeId	uuid	Yes	Service reference
ItemType	varchar(15)	Yes	BasicSalary, Component, Bonus, Overtime
EntryType	varchar(10)	Yes	Earning, Deduction

Field	Type (PostgreSQL)	Required	Constraints / notes
Description	varchar(200)	Yes	
Amount	numeric(12,2)	Yes	0 or greater
SourceId	uuid	No	Internal reference to component, bonus or overtime row (same database)

Payslip (SalaryDB.Payslips)

Field	Type (PostgreSQL)	Required	Constraints / notes
PayslipId	uuid	Yes	Primary key
PayrollId	uuid	Yes	Foreign key to Payrolls (same database)
EmployeeId	uuid	Yes	Service reference; unique together with PayrollId
TotalEarnings / TotalDeductions / NetPay	numeric(12,2)	Yes	Snapshot calculated at generation
GeneratedAt	timestampz	Yes	Server-set (UTC)

Leave (LeaveDB.Leaves)

Field	Type (PostgreSQL)	Required	Constraints / notes
LeaveId	uuid	Yes	Primary key
EmployeeId	uuid	Yes	Service reference
LeaveType	varchar(30)	Yes	One of the configured leave types
StartDate / EndDate	date	Yes	StartDate less than or equal to EndDate
Reason	varchar(500)	Yes	
Status	varchar(10)	Yes	Pending, Approved, Rejected
AppliedAt	timestampz	Yes	Server-set (UTC)
ApprovedBy / ApprovedAt	uuid / timestampz	No	Set on approval or rejection; ApprovedBy is a service reference
CreatedAt / UpdatedAt	timestampz	Yes	Server-set (UTC)

Attendance (AttendanceDB.AttendanceRecords)

Field	Type (PostgreSQL)	Required	Constraints / notes
AttendanceId	uuid	Yes	Primary key
EmployeeId	uuid	Yes	Service reference
AttendanceDate	date	Yes	EmployeeId + AttendanceDate unique
CheckInTime	timestampz	Yes	Server-set
CheckOutTime	timestampz	No	Not earlier than CheckInTime
Status	varchar(12)	Yes	CheckedIn, CheckedOut
CreatedAt / UpdatedAt	timestampz	Yes	Server-set (UTC)

SupportTicket (SupportDB.SupportTickets)

Field	Type (PostgreSQL)	Required	Constraints / notes
TicketId	uuid	Yes	Primary key
EmployeeId	uuid	Yes	Service reference (ticket owner)
Title	varchar(150)	Yes	
Description	varchar(2000)	Yes	
Priority	varchar(10)	Yes	Low, Medium, High
Status	varchar(12)	Yes	Open, Assigned, InProgress, Resolved, Closed
AssignedTo	uuid	No	Service reference to a SupportStaff employee

Field	Type (PostgreSQL)	Required	Constraints / notes
CreatedAt / UpdatedAt	timestamptz	Yes	Server-set (UTC)
ResolvedAt	timestamptz	No	Set when Resolved

Notification (NotificationDB.Notifications)

Field	Type (PostgreSQL)	Required	Constraints / notes
NotificationId	uuid	Yes	Primary key
EmployeeId	uuid	Yes	Service reference (recipient)
Title	varchar(150)	Yes	
Message	varchar(500)	Yes	
Type	varchar(30)	Yes	LeaveApproved, LeaveRejected, SalaryUpdated, OvertimeApproved, SupportTicketUpdated
IsRead	boolean	Yes	Default false
CreatedAt	timestamptz	Yes	Server-set (UTC)

ProcessedEvent (NotificationDB.ProcessedEvents)

Field	Type (PostgreSQL)	Required	Constraints / notes
EventId	uuid	Yes	Primary key; supports idempotent consumption
ProcessedAt	timestamptz	Yes	Server-set (UTC)

18. Database and Normalization Requirements

18.1 Database Requirements

ID	Requirement
DB-001	Each microservice shall have its own PostgreSQL database (database-per-service) as listed in ARC-004.
DB-002	A service shall not connect to, query or write to another service's database.
DB-003	Cross-service identifiers (EmployeeId, DepartmentId, CreatedBy, ReviewedBy, ApprovedBy, GeneratedBy, AssignedTo) shall be stored as plain columns. No physical foreign key shall be created between databases of different services.
DB-004	Relationships inside one service database shall use foreign keys (for example SalaryId in SalaryComponents and SalaryRevisions; PayrollId in PayrollItems and Payslips).
DB-005	Uniqueness shall be enforced by database constraints in addition to application checks: Employees.Email; Departments.DepartmentName; AttendanceRecords (EmployeeId, AttendanceDate); EmployeeSalaries.EmployeeId; Payrolls (PayrollMonth, PayrollYear); Payslips (PayrollId, EmployeeId); SalaryComponents (SalaryId, ComponentName).
DB-006	Schema changes shall be managed through Entity Framework Core migrations kept in each service's source.
DB-007	Columns used for filtering and joining shall be indexed, including Email, DepartmentId, EmployeeId, Status, AttendanceDate and CreatedAt where they are used in queries.
DB-008	Operations that change more than one row or table (salary revision, payroll generation, payslip generation, overtime inclusion) shall run in a single database transaction.
DB-009	Amounts shall be stored as numeric(12,2); timestamps as timestamptz in UTC.

18.2 Normalization Requirements

ID	Requirement
DB-010	Each service schema shall be in at least third normal form (3NF): one fact per column, no repeating groups, and every non-key column dependent only on the key.
DB-011	Repeating data shall be modelled as child tables, not columns: salary components in SalaryComponents, salary history in SalaryRevisions, payroll lines in PayrollItems.

ID	Requirement
DB-012	Values that can be derived (for example working hours, department employee count, leave duration) shall not be stored.
DB-013	Controlled exception: Payslips store TotalEarnings, TotalDeductions and NetPay as a snapshot at generation, and SalaryRevisions store PreviousAmount, so that historical records remain correct when current salary data changes. This exception is intentional and documented.

18.3 Database Summary

Database	Owner	Tables	Key constraints	Service references held
EmployeeDB	Employee Service	Employees	PK EmployeeId; UNIQUE Email	DepartmentId
DepartmentDB	Department Service	Departments	PK DepartmentId; UNIQUE DepartmentName	–
SalaryDB	Salary Service	EmployeeSalaries, SalaryComponents, SalaryRevisions, Bonuses, OvertimeRequests, Payrolls, PayrollItems, Payslips	FKs inside SalaryDB; UNIQUE EmployeeId; UNIQUE (PayrollMonth, PayrollYear); UNIQUE (PayrollId, EmployeeId)	EmployeeId, CreatedBy, ReviewedBy, GeneratedBy
LeaveDB	Leave Service	Leaves	PK LeaveId; CHECK StartDate <= EndDate	EmployeeId, ApprovedBy
AttendanceDB	Attendance Service	AttendanceRecords	PK AttendanceId; UNIQUE (EmployeeId, AttendanceDate); CHECK CheckOutTime >= CheckInTime	EmployeeId
SupportDB	Support Service	SupportTickets	PK TicketId	EmployeeId, AssignedTo
NotificationDB	Notification Service	Notifications, ProcessedEvents	PK NotificationId; PK EventId	EmployeeId

The Security component keeps its own credential store (accounts, password hashes, role assignments, reset tokens). It is separate from the seven service databases and is not accessible to other services.

19. API Requirements

19.1 General API Requirements

ID	Requirement
API-001	All APIs shall be RESTful over HTTPS, exchanging JSON (UTF-8), and shall be exposed only through the API Gateway.
API-002	All endpoints shall be versioned with a /api/v1 prefix (see Section 25). Resource paths: /api/v1/employees, /api/v1/departments, /api/v1/salaries, /api/v1/leaves, /api/v1/attendance, /api/v1/support-tickets, /api/v1/notifications; authentication and user management use /api/v1/auth and /api/v1/users.
API-003	HTTP methods shall be used consistently: GET reads, POST creates or starts an action, PUT updates or changes state, DELETE removes.
API-004	Success codes: 200 for reads and updates, 201 for creation (with resource location), 202 for accepted password-reset requests, 204 for delete, deactivate and password reset. Error codes are defined in Section 28.
API-005	List endpoints shall support paging (pageNumber, pageSize) and return totalCount; filters shall be query parameters.
API-006	Every request except login, forgot-password and reset-password shall carry a bearer access token.
API-007	Request and response field names shall match the names in Section 17 (camelCase in JSON, for example employeeId, departmentId).
API-008	Server-controlled fields (identifiers, CreatedAt, UpdatedAt, server-set Status, ResolvedAt, AppliedAt) shall never be accepted from the client.
API-009	Self-service endpoints use the route segment "me" or the token identity; they never accept an EmployeeId that differs from the token.

ID	Requirement
API-010	A Bruno collection and a Swagger document shall exist for every endpoint listed below.

19.2 API Summary Table

Method	Endpoint	Purpose	Access	FR
POST	/api/v1/auth/login	Login	Anonymous	FR-AUT-001
POST	/api/v1/auth/forgot-password	Request Password Reset	Anonymous	FR-AUT-002
POST	/api/v1/auth/reset-password	Reset Password	Anonymous	FR-AUT-003
POST	/api/v1/users	Manage User Accounts	Admin	FR-AUT-004
GET	/api/v1/users	Manage User Accounts	Admin	FR-AUT-004
GET	/api/v1/users/{userId}	Manage User Accounts	Admin	FR-AUT-004
PUT	/api/v1/users/{userId}	Manage User Accounts	Admin	FR-AUT-004
PUT	/api/v1/users/{userId}/deactivate	Manage User Accounts	Admin	FR-AUT-004
PUT	/api/v1/users/{userId}/role	Assign User Role	Admin	FR-AUT-005
POST	/api/v1/employees	Add Employee	HR, Admin	FR-EMP-001
GET	/api/v1/employees/{employeeId}	View Employee	HR, Admin: all. Manager: team. Others: own	FR-EMP-002
GET	/api/v1/employees	View All Employees	HR, Admin	FR-EMP-003
GET	/api/v1/employees/search	Search Employees	HR, Admin: all. Manager: team	FR-EMP-004
PUT	/api/v1/employees/{employeeId}	Update Employee	HR, Admin	FR-EMP-005
DELETE	/api/v1/employees/{employeeId}	Delete Employee	HR, Admin	FR-EMP-006
GET	/api/v1/employees/me	View Own Profile	All roles (own data)	FR-EMP-007
PUT	/api/v1/employees/me	Update Own Profile	All roles (own data)	FR-EMP-008
GET	/api/v1/employees/team	View Team Employees	Manager	FR-EMP-009
POST	/api/v1/departments	Add Department	HR, Admin	FR-DEP-001
GET	/api/v1/departments/{departmentId}	View Department	All authenticated roles	FR-DEP-002
GET	/api/v1/departments	View All Departments	All authenticated roles	FR-DEP-003
PUT	/api/v1/departments/{departmentId}	Update Department	HR, Admin	FR-DEP-004
DELETE	/api/v1/departments/{departmentId}	Delete Department	HR, Admin	FR-DEP-005
POST	/api/v1/salaries	Add Salary	HR, Admin	FR-SAL-001
GET	/api/v1/salaries/me	View Salary	HR, Admin: all. Others: own	FR-SAL-002
GET	/api/v1/salaries/{salaryId}	View Salary	HR, Admin: all. Others: own	FR-SAL-002
PUT	/api/v1/salaries/{salaryId}	Update Salary	HR, Admin	FR-SAL-003
POST	/api/v1/salaries/{salaryId}/components	Add Salary Component	HR, Admin	FR-SAL-004
PUT	/api/v1/salaries/{salaryId}/components/{componentId}	Update Salary Component	HR, Admin	FR-SAL-005
GET	/api/v1/salaries/me/components	View Salary Components	HR, Admin: all. Others: own	FR-SAL-006
GET	/api/v1/salaries/{salaryId}/components	View Salary Components	HR, Admin: all. Others: own	FR-SAL-006
POST	/api/v1/salaries/{salaryId}/revisions	Create Salary Revision	HR, Admin	FR-SAL-007
GET	/api/v1/salaries/me/revisions	View Salary Revision History	HR, Admin: all. Others: own	FR-SAL-008
GET	/api/v1/salaries/{salaryId}/revisions	View Salary Revision History	HR, Admin: all. Others: own	FR-SAL-008
POST	/api/v1/salaries/bonuses	Add Bonus	HR, Admin	FR-SAL-009

Method	Endpoint	Purpose	Access	FR
PUT	/api/v1/salaries/bonuses/{bonusId}	Update Bonus	HR, Admin	FR-SAL-010
GET	/api/v1/salaries/bonuses	View Bonus	HR, Admin: all. Others: own, Approved only	FR-SAL-011
POST	/api/v1/salaries/overtime	Submit Overtime Request	All roles (own data)	FR-SAL-012
GET	/api/v1/salaries/overtime	View Overtime Requests	HR, Admin: all. Manager: team. Others: own	FR-SAL-013
PUT	/api/v1/salaries/overtime/{overtimeId}/approve	Approve Overtime	Manager: team. HR, Admin	FR-SAL-014
PUT	/api/v1/salaries/overtime/{overtimeId}/reject	Reject Overtime	Manager: team. HR, Admin	FR-SAL-015
POST	/api/v1/salaries/payrolls/{payrollId}/overtime	Include Approved Overtime in Payroll	HR, Admin	FR-SAL-016
POST	/api/v1/salaries/payrolls	Generate Payroll	HR, Admin	FR-SAL-017
GET	/api/v1/salaries/payrolls	View Payroll	HR, Admin	FR-SAL-018
GET	/api/v1/salaries/payrolls/{payrollId}	View Payroll	HR, Admin	FR-SAL-018
GET	/api/v1/salaries/payrolls/{payrollId}/items	View Payroll Items	HR, Admin	FR-SAL-019
POST	/api/v1/salaries/payrolls/{payrollId}/payslips	Generate Payslip	HR, Admin	FR-SAL-020
GET	/api/v1/salaries/payslips	View Payslip	HR, Admin: all. Others: own	FR-SAL-021
GET	/api/v1/salaries/payslips/{payslipId}	View Payslip	HR, Admin: all. Others: own	FR-SAL-021
POST	/api/v1/leaves	Apply Leave	All roles (own data)	FR-LEV-001
GET	/api/v1/leaves/{leaveId}	View Leave	HR, Admin: all. Manager: team. Others: own	FR-LEV-002
GET	/api/v1/leaves/history	View Leave History	All roles (own data)	FR-LEV-003
GET	/api/v1/leaves/requests	View Leave Requests	HR, Admin: all. Manager: team	FR-LEV-004
PUT	/api/v1/leaves/{leaveId}/approve	Approve Leave	Manager: team. HR	FR-LEV-005
PUT	/api/v1/leaves/{leaveId}/reject	Reject Leave	Manager: team. HR	FR-LEV-006
POST	/api/v1/attendance/check-in	Check In	All roles (own data)	FR-ATT-001
POST	/api/v1/attendance/check-out	Check Out	All roles (own data)	FR-ATT-002
GET	/api/v1/attendance/me	View Attendance	All roles (own data)	FR-ATT-003
GET	/api/v1/attendance/history	View Attendance History	All roles (own data)	FR-ATT-004
GET	/api/v1/attendance/team	View Team Attendance	HR, Admin: all. Manager: team	FR-ATT-005
POST	/api/v1/support-tickets	Create Support Ticket	All roles (own data)	FR-SUP-001
GET	/api/v1/support-tickets/{ticketId}	View Ticket	Support Staff, Admin: all. Others: own	FR-SUP-002
GET	/api/v1/support-tickets	View Tickets	Support Staff, Admin: all. Others: own	FR-SUP-003
PUT	/api/v1/support-tickets/{ticketId}/assign	Assign Ticket	Support Staff, Admin	FR-SUP-004
PUT	/api/v1/support-tickets/{ticketId}/status	Update Ticket Status	Support Staff (assignee), Admin	FR-SUP-005
PUT	/api/v1/support-tickets/{ticketId}/resolve	Resolve Ticket	Support Staff (assignee), Admin	FR-SUP-006
PUT	/api/v1/support-tickets/{ticketId}/close	Close Ticket	Support Staff (assignee), Admin	FR-SUP-007
Kafka	Consumer (no REST endpoint)	Create Notification from Event	System	FR-NOT-001
GET	/api/v1/notifications	View Notifications	All roles (own data)	FR-NOT-002
PUT	/api/v1/notifications/{notificationId}/read	Mark Notification as Read	All roles (own data)	FR-NOT-003

Total: 70 operations across 61 requirements. Access shows the roles or scopes allowed; "Others" means any authenticated role not named.

20. Communication Requirements

20.1 Synchronous and Asynchronous Communication

ID	Requirement
COM-001	Synchronous, request-response communication that needs an immediate result shall use REST over HTTP(S), through the Eureka registry.
COM-002	Asynchronous notification of business events shall use Apache Kafka. Kafka shall not be used for operations that need an immediate response.
COM-003	A service that must know a fact owned by another service (for example that a DepartmentId exists) shall ask that service through its REST API.
COM-004	Service-to-service REST calls shall carry a service credential and the correlation ID, and shall use the same versioned paths as external clients.
COM-005	Every outbound REST call shall have a timeout, a limited number of retries with back-off for safe (GET) calls, and a circuit breaker.
COM-006	If a required dependency cannot be reached, the operation shall fail with 503 DEPENDENCY_UNAVAILABLE and shall not store a partial change.
COM-007	Failure of the Notification Service or Kafka shall not prevent business operations from completing; events shall be delivered when the consumer recovers (see KAF-006).

20.2 Synchronous Service-to-Service Calls

Caller	Callee	Purpose	Requirement
Employee Service	Department Service	Verify that DepartmentId exists	FR-EMP-001, FR-EMP-005
Department Service	Employee Service	Count employees assigned to DepartmentId before deletion	FR-DEP-005
Salary Service	Employee Service	Verify EmployeeId; verify Active status for payroll; resolve team for overtime review	FR-SAL-001, 009, 014, 015, 017
Leave Service	Employee Service	Resolve team membership for review and approval	FR-LEV-004, 005, 006
Attendance Service	Employee Service	Resolve team membership for team attendance	FR-ATT-005
Support Service	Employee Service	Verify assignee exists and has Role SupportStaff	FR-SUP-004
Security component	Employee Service	Verify EmployeeId when creating a user account	FR-AUT-004

21. Kafka / Event Requirements

21.1 Event Requirements

ID	Requirement
KAF-001	The system shall use Apache Kafka for the asynchronous events LeaveApproved, LeaveRejected, SalaryUpdated, OvertimeApproved and SupportTicketUpdated, and for no synchronous purpose.
KAF-002	The owning service shall publish an event only after the corresponding state change has been committed. The change and its publication shall not be able to diverge (for example, by using a transactional outbox).
KAF-003	Every event shall use a common envelope: EventId (UUID), EventType, EventVersion, OccurredAt (UTC), CorrelationId and Payload.
KAF-004	Events shall carry identifiers and non-sensitive attributes only. Salary, bonus and payslip amounts shall never be included (BR-SAL-014).
KAF-005	Events shall use EmployeeId as the message key so that events of one employee are processed in order.
KAF-006	Delivery is at-least-once. The Notification Service shall be an idempotent consumer using EventId (BR-NOT-003).
KAF-007	A message that cannot be processed after a configured number of retries with back-off shall be sent to a dead-letter topic and logged with its correlation ID.
KAF-008	Each consuming service shall use its own consumer group. Only the Notification Service consumes the events defined here.
KAF-009	Topic names and payload schemas shall be documented and versioned through EventVersion; changes shall be backward compatible within a version.
KAF-010	The correlation ID shall be carried in Kafka message headers and included in consumer log entries.

21.2 Kafka Event Summary

Event	Topic	Producer	Triggered by	Payload (besides envelope)	Notification created
LeaveApproved	leave-approved	Leave Service	FR-LEV-005	LeaveId, EmployeeId, LeaveType, StartDate, EndDate, ApprovedBy	Employee: "Leave approved"
LeaveRejected	leave-rejected	Leave Service	FR-LEV-006	LeaveId, EmployeeId, LeaveType, StartDate, EndDate, ApprovedBy	Employee: "Leave rejected"
SalaryUpdated	salary-updated	Salary Service	FR-SAL-001, 003, 004, 005, 007	SalaryId, EmployeeId, ChangeType (Created, Updated, ComponentAdded, ComponentUpdated, Revised), UpdatedBy	Employee: "Salary updated"
OvertimeApproved	overtime-approved	Salary Service	FR-SAL-014	OvertimeId, EmployeeId, OvertimeDate, Hours, ApprovedBy	Employee: "Overtime approved"
SupportTicketUpdated	support-ticket-updated	Support Service	FR-SUP-004, 005, 006, 007	TicketId, EmployeeId, AssignedTo, Status, UpdatedBy	Ticket owner: "Support ticket updated"; assignee (on assignment): "Support ticket updated"

21.3 Event-to-Notification Mapping

Event	Notification Type	Recipient EmployeeId	Title	Message (template)
LeaveApproved	LeaveApproved	Payload.EmployeeId	Leave approved	Your leave from {StartDate} to {EndDate} has been approved.
LeaveRejected	LeaveRejected	Payload.EmployeeId	Leave rejected	Your leave from {StartDate} to {EndDate} has been rejected.
SalaryUpdated	SalaryUpdated	Payload.EmployeeId	Salary updated	Your salary information has been updated.
OvertimeApproved	OvertimeApproved	Payload.EmployeeId	Overtime approved	Your overtime request for {OvertimeDate} has been approved.
SupportTicketUpdated	SupportTicketUpdated	Payload.EmployeeId; and Payload.AssignedTo when the ticket was just assigned	Support ticket updated	Ticket {TicketId} is now {Status}.

The "payroll updates" tab shown in the notification wireframe is served by `SalaryUpdated` and `OvertimeApproved` notifications; no additional event is introduced.

22. Security Requirements

Security requirements use the identifier prefix `NFR-SEC` and are part of the non-functional requirements of Section 27.

22.1 Authentication

ID	Requirement
NFR-SEC-001	Every request except login, forgot-password and reset-password shall be authenticated with a signed, time-limited bearer access token issued by the Security component. The token shall carry <code>UserId</code> , <code>EmployeeId</code> , role and expiry.
NFR-SEC-002	The API Gateway shall validate the token signature, issuer and expiry before routing; each service shall validate the token again before authorizing (defence in depth).
NFR-SEC-003	Services shall trust the identity and role in the token only; they shall not accept an identity or role supplied in the request body or query string.

22.2 Password Handling

ID	Requirement
NFR-SEC-004	Passwords shall be stored only as salted hashes produced by a slow, adaptive password-hashing algorithm; plain-text or reversible storage is prohibited. Reset tokens shall be stored as hashes, be single-use and expire after 30 minutes.
NFR-SEC-005	Password policy: at least 8 characters with upper-case, lower-case, digit and special character.
NFR-SEC-006	Login shall lock an account temporarily after 5 consecutive failed attempts (configurable) and shall return the same generic message for every authentication failure.

22.3 Authorization

ID	Requirement
NFR-SEC-007	Authorization shall be role-based (RBAC) as defined in the Role Permission Matrix (Section 9.3), enforced on the server for every endpoint. Hiding a control in the frontend is never a substitute.
NFR-SEC-008	Resource-level authorization shall be enforced in addition to roles: an actor may access employee-scoped data only within the scope <code>Own</code> , <code>Team</code> or <code>All</code> that their role allows (BR-GEN-002).
NFR-SEC-009	An employee shall never be able to read or change another employee's private information (profile, salary, bonus, payslip, leave, attendance, tickets, notifications) unless the role explicitly permits it.
NFR-SEC-010	Employees shall not be able to create, modify or delete salary, bonus or salary revision data through any endpoint (BR-SAL-001).
NFR-SEC-011	Access denials shall return 403 FORBIDDEN and shall be logged with the actor, resource type and endpoint, without sensitive values.

22.4 Sensitive Data and Secure API Access

ID	Requirement
NFR-SEC-012	Salary, bonus, payroll and payslip data shall be returned only to authorized actors, shall not appear in logs or Kafka events, and shall not be exposed in list responses of other services.
NFR-SEC-013	All external traffic shall use HTTPS (TLS). Internal service-to-service traffic shall be authenticated with service credentials.
NFR-SEC-014	Services shall be reachable only through the API Gateway from outside the internal network; databases and Kafka shall not be reachable from clients.
NFR-SEC-015	Cross-origin requests shall be limited to the configured frontend origin(s).
NFR-SEC-016	All input shall be validated on the server (Section 28). Data access shall use parameterized queries through Entity Framework Core; user input shall never be concatenated into SQL.
NFR-SEC-017	Error responses shall be safe: no stack traces, SQL, connection strings, file paths or internal names (NFR-ERR-002).

ID	Requirement
NFR-SEC-018	Passwords, password hashes, access tokens, reset tokens and secrets shall never be written to logs (NFR-LOG-006), source control or error responses.
NFR-SEC-019	Secrets (database credentials, signing keys, Kafka credentials) shall be supplied through environment or secret configuration, not source code.

23. API Gateway Requirements

ID	Requirement
GW-001	An API Gateway shall be the single entry point for all client requests (Client > API Gateway > Microservices).
GW-002	The gateway shall route each versioned path to the correct service: /api/v1/employees to Employee Service, /api/v1/departments to Department Service, /api/v1/salaries to Salary Service, /api/v1/leaves to Leave Service, /api/v1/attendance to Attendance Service, /api/v1/support-tickets to Support Service, /api/v1/notifications to Notification Service, and /api/v1/auth and /api/v1/users to the Security component.
GW-003	The gateway shall resolve service locations through Eureka and shall balance load across registered instances.
GW-004	The gateway shall reject requests without a valid token (401) on all routes except login, forgot-password and reset-password.
GW-005	The gateway shall generate a correlation ID for each request when none is supplied, forward it to services and return it in the response.
GW-006	The gateway shall return the standard error structure with 503 DEPENDENCY_UNAVAILABLE when a target service is unavailable, and 404 NOT_FOUND for unknown routes or API versions.
GW-007	The gateway shall enforce HTTPS, configured CORS origins and a maximum request size.
GW-008	The gateway shall log each request (method, path, status, duration, correlation ID) without request bodies or credentials.
GW-009	The gateway should apply request rate limiting to the login and password-reset routes.

24. Service Discovery Requirements

ID	Requirement
SD-001	Every microservice and the API Gateway shall register with the Eureka server at start-up using a logical service name (for example employee-service, department-service).
SD-002	Services shall send periodic heartbeats and be removed from the registry when heartbeats stop.
SD-003	Service-to-service REST calls and gateway routing shall use logical service names resolved through Eureka; no hard-coded host names or ports.
SD-004	Each service shall expose a health endpoint (liveness and readiness) used by the registry and by monitoring; a service reporting not-ready shall not receive traffic.
SD-005	Multiple instances of a service shall be able to register under the same logical name.
SD-006	Registry unavailability shall not stop running services from serving requests that use cached registry data.

25. API Versioning Requirements

ID	Requirement
VER-001	All endpoints shall be versioned in the URL path with the prefix /api/v1, for example /api/v1/employees, /api/v1/departments, /api/v1/salaries, /api/v1/leaves, /api/v1/attendance, /api/v1/support-tickets, /api/v1/notifications.
VER-002	A breaking change (removed or renamed field, changed meaning, changed status code) shall be released under a new version (/api/v2) and shall not alter v1.
VER-003	Non-breaking additions (new optional fields or endpoints) may be added to the current version.
VER-004	When a new version is introduced, the previous version shall remain available for a documented deprecation period, and deprecated endpoints shall be marked in Swagger.
VER-005	The gateway shall route by version prefix; a request for an unsupported version shall return 404 NOT_FOUND.
VER-006	Kafka event versions shall be managed separately through EventVersion (KAF-009).

26. Central Logging Requirements

ID	Requirement
NFR-LOG-001	All microservices, the API Gateway and the Security component shall send their logs to a central logging platform where they can be searched together.
NFR-LOG-002	Logs shall be structured (for example JSON) and include: UTC timestamp, level, service name, environment, correlation ID, message and, for errors, exception details.
NFR-LOG-003	A correlation ID shall be created at the gateway and propagated through REST headers and Kafka message headers; it shall appear in every log entry of the request and as traceId in error responses.
NFR-LOG-004	Standard levels (Trace, Debug, Information, Warning, Error, Critical) shall be used; the production default is Information. Levels shall be changeable without redeployment.
NFR-LOG-005	The following shall be logged: request summary (method, path, status, duration); authentication success and failure; authorization denials; business state changes (leave, overtime, salary, ticket, payroll actions with actor and entity identifiers); Kafka publish, consume and dead-letter events; failed inter-service calls; unhandled exceptions.
NFR-LOG-006	The following shall never be logged: passwords, password hashes, reset tokens, access tokens, secrets, and salary, bonus, payroll or payslip amounts. Personal data beyond identifiers shall be masked.
NFR-LOG-007	Logs shall be retained for at least 30 days (configurable).
NFR-LOG-008	Logs shall be searchable by correlation ID, service, level and time range.
NFR-LOG-009	A failure of the logging system shall not cause a request to fail.

27. Non-Functional Requirements

Security, logging and error-handling requirements are defined in Sections 22, 26 and 28 respectively. The values below are baseline targets for the project deployment (Assumption A-11) and are verified as described in Section 29.

27.1 Performance

ID	Requirement
NFR-PERF-001	At least 95% of single-resource GET requests shall complete within 500 ms at the gateway under normal load (up to 100 concurrent users).
NFR-PERF-002	At least 95% of list and search requests (page size up to 100) shall complete within 1 second under normal load.
NFR-PERF-003	At least 95% of create, update and state-change requests shall complete within 1 second, excluding asynchronous event processing.
NFR-PERF-004	Generating a payroll for up to 500 employees shall complete within 60 seconds.
NFR-PERF-005	A notification shall be available to the recipient within 5 seconds of the event being published under normal load.
NFR-PERF-006	List endpoints shall be paged; unbounded queries are not permitted.

27.2 Scalability

ID	Requirement
NFR-SCAL-001	Services shall be stateless so that additional instances can be started behind the gateway without code changes.
NFR-SCAL-002	Each service shall be scalable independently of the others, including its database.
NFR-SCAL-003	Kafka topics shall be partitioned by EmployeeId so that consumer instances can be added to a consumer group.
NFR-SCAL-004	The baseline deployment shall support 500 employees and 100 concurrent users without redesign.

27.3 Availability

ID	Requirement
NFR-AVL-001	The target availability of the deployed system shall be 99% per month excluding planned maintenance.
NFR-AVL-002	Failure of one service shall not stop unrelated services (fault isolation).
NFR-AVL-003	If the Notification Service or a consumer is down, business operations shall continue and events shall be retained in Kafka until consumed.
NFR-AVL-004	Every service shall expose liveness and readiness health endpoints (SD-004).

27.4 Reliability

ID	Requirement
NFR-REL-001	Committed state changes shall not lose their events (KAF-002); consumers shall be idempotent (KAF-006).
NFR-REL-002	Multi-row operations shall be atomic (DB-008): salary revision, payroll generation, payslip generation, overtime inclusion.
NFR-REL-003	Concurrent modification of the same record (for example two approvers deciding the same leave) shall result in exactly one successful decision and a 409 CONFLICT for the other.
NFR-REL-004	Databases shall be backed up on a schedule, and restoration shall be documented and tested at least once.

27.5 Maintainability

ID	Requirement
NFR-MNT-001	Each service shall follow the same project structure, naming and coding conventions and separate controller, business and data-access layers (ARC-009).
NFR-MNT-002	Configuration shall be externalized (ARC-012); schema changes shall use migrations (DB-006).
NFR-MNT-003	Each service repository shall contain a README describing how to build, run and test the service.
NFR-MNT-004	OpenAPI documentation shall be generated from the code so that it cannot diverge from the implementation.

27.6 Usability

ID	Requirement
NFR-USB-001	The frontend shall follow the wireframe layout (left navigation, top header, workspace, tables, cards, status badges, action buttons) consistently on every screen.
NFR-USB-002	Validation and error messages shall be human-readable and shown next to the field concerned.
NFR-USB-003	Navigation shall show only the screens and actions permitted for the signed-in role.
NFR-USB-004	The frontend shall be usable in current desktop browsers at a width of 1280 px or more.

27.7 Testability

ID	Requirement
NFR-TST-001	Business logic shall be testable without a database, Kafka or network (dependencies injected through interfaces).
NFR-TST-002	Unit tests of the business layer shall reach at least 70% line coverage.
NFR-TST-003	Bruno collections shall run from the command line, use environment variables for base URL and tokens, and be repeatable on a freshly seeded database.

ID	Requirement
NFR-TST-004	Test data for every role (Admin, HR, Manager, Employee, Support Staff) shall be available through a documented seed procedure.

27.8 Requirements Defined Elsewhere

Category	Identifiers	Section
Security	NFR-SEC-001 to NFR-SEC-019	22
Logging	NFR-LOG-001 to NFR-LOG-009	26
Error handling	NFR-ERR-001 to NFR-ERR-008	28

28. Validation and Error Handling

28.1 Validation Rules

ID	Requirement
VAL-001	Every request shall be validated on the server, whatever validation the client performs.
VAL-002	Required fields shall be present and not blank after trimming.
VAL-003	String lengths shall not exceed the limits in Section 17.
VAL-004	E-mail shall be a valid address; phone shall meet BR-EMP-003.
VAL-005	Dates shall be valid ISO 8601 dates; StartDate shall be less than or equal to EndDate (BR-LEV-001); check-out time shall not be earlier than check-in time (BR-ATT-002).
VAL-006	Enumeration values (Role, EmploymentStatus, Priority, Status, ComponentType, LeaveType) shall be one of the defined values.
VAL-007	Identifiers in routes and bodies shall be valid UUIDs; a malformed identifier returns 400 VALIDATION_ERROR.
VAL-008	Monetary amounts and overtime hours shall meet BR-SAL-007 and BR-SAL-008.
VAL-009	pageNumber shall be 1 or greater; pageSize shall be between 1 and 100.
VAL-010	Server-controlled fields supplied by a client shall be ignored or rejected and never applied (API-008).
VAL-011	Uniqueness, state-transition and dependency violations shall return 409 CONFLICT (for example duplicate Email, illegal ticket transition, department with employees).

28.2 Error Handling Requirements

ID	Requirement
NFR-ERR-001	All services and the gateway shall return errors in one structure (compatible with RFC 7807): status, code, message, traceId, timestamp and, for validation errors, an errors list of {field, message}.
NFR-ERR-002	Error responses shall not contain stack traces, SQL, connection strings, file paths, class names or internal host names.
NFR-ERR-003	Unhandled exceptions shall be caught by global handling, logged with the correlation ID, and returned as 500 INTERNAL_ERROR with a generic message.
NFR-ERR-004	HTTP status codes shall be used as defined in the error catalogue below.
NFR-ERR-005	401 shall be used for missing or invalid credentials and 403 for authenticated users who lack permission; authentication failure messages shall be generic.
NFR-ERR-006	A validation failure shall report all field errors in a single response.
NFR-ERR-007	When a required dependency is unavailable, the operation shall fail with 503 DEPENDENCY_UNAVAILABLE and shall leave no partial change.
NFR-ERR-008	Concurrency conflicts shall return 409 CONFLICT.

28.3 Error Catalogue

Code	HTTP status	Used when
VALIDATION_ERROR	400	Malformed request, missing or invalid field, invalid enumeration, paging or date value.

Code	HTTP status	Used when
REFERENCE_NOT_FOUND	400	A referenced EmployeeId, DepartmentId or assignee in the request body does not exist.
UNAUTHENTICATED	401	Token missing, invalid or expired; login failed.
FORBIDDEN	403	Role or scope does not permit the operation, including access to another employee's private data.
NOT_FOUND	404	The resource identified in the route does not exist.
CONFLICT	409	Duplicate value, state conflict, dependency conflict, or concurrent modification (includes invalid state transitions).
INTERNAL_ERROR	500	Unexpected server error. Only a generic message and the traceId are returned.
DEPENDENCY_UNAVAILABLE	503	A required service or infrastructure component could not be reached; no partial change is stored.

29. Testing Requirements

29.1 Test Types

Test type	Tool / approach	Scope
Unit testing	xUnit	Business rules, validators, state transitions, mappers and services with mocked dependencies.
API testing	Bruno	Every endpoint of every service through the API Gateway: success, validation, authorization and error cases.
Integration testing	xUnit with test PostgreSQL and Kafka instances	Service with real database; REST calls between services; Kafka publish and consume; Eureka registration; gateway routing.
Validation testing	xUnit and Bruno	Every rule of Section 28 and every field constraint in Section 17.
Authorization testing	Bruno (role-based tokens) and xUnit	Every role against every endpoint of the Role Permission Matrix, including Own and Team scope.
Error handling testing	Bruno and xUnit	Error structure, status codes, absence of internal details, dependency failure and dead-letter handling.

29.2 Testing Requirements

ID	Requirement
TR-001	Every functional requirement shall be covered by at least one automated test (xUnit or Bruno), recorded in the traceability matrix (Appendix A).
TR-002	Unit tests shall use xUnit and shall cover every business rule (BR-xxx) enforced in the service.
TR-003	Each service shall have a Bruno collection with folders per resource, environment variables for each role's token, and assertions on status code and body.
TR-004	Authorization tests shall verify both the permitted and the forbidden outcome for each role in the Role Permission Matrix, including access to another employee's data.
TR-005	Integration tests shall verify each Kafka event from publication to notification creation, including duplicate delivery.
TR-006	Validation tests shall cover missing, malformed, too-long and out-of-range values for every input field.
TR-007	Error handling tests shall verify status codes, the error structure, and that no internal detail is exposed.
TR-008	Security tests shall verify that passwords, tokens and salary amounts are absent from logs and events.
TR-009	Performance targets (NFR-PERF-001 to 005) shall be measured once on the integrated system.
TR-010	All tests shall be runnable from the command line and shall pass before a release is accepted.

29.3 Key Test Scenarios

ID	Scenario	Test type	Requirement
TS-01	Add employee with an e-mail that already exists returns 409.	UT, API	FR-EMP-001
TS-02	Add employee with a missing required field or malformed e-mail returns 400 with field-level errors.	UT, API, Validation	FR-EMP-001

ID	Scenario	Test type	Requirement
TS-03	An Employee requests another employee's record and receives 403; a Manager receives 403 for a non-team employee and 200 for a team member.	API, Authorization	FR-EMP-002
TS-04	Delete a department that has assigned employees returns 409; delete of an empty department returns 204.	IT, API	FR-DEP-005
TS-05	An Employee calling any salary, component, revision or bonus write endpoint receives 403.	API, Authorization	FR-SAL-001, 003, 004, 005, 007, 009, 010
TS-06	A Manager or Employee requesting another employee's salary, payslip or bonus receives 403.	API, Authorization	FR-SAL-002, 011, 021
TS-07	Creating a salary revision stores the revision, captures the previous amount, updates BaseSalary and publishes SalaryUpdated; revisions cannot be edited.	UT, IT	FR-SAL-007
TS-08	Overtime flow: Employee submits, Manager approves, OvertimeApproved is published and a notification exists for the Employee.	IT, API	FR-SAL-012, 014, NOT-001
TS-09	Approving or rejecting overtime that is not Pending returns 409; approving own overtime returns 403.	UT, API	FR-SAL-014, 015
TS-10	An approved overtime request can be included in a Generated payroll once; a second inclusion returns 409.	IT, API	FR-SAL-016
TS-11	Generating payroll for an existing period returns 409; generating payslips twice returns 409.	IT, API	FR-SAL-017, 020
TS-12	Apply leave with StartDate after EndDate returns 400; equal dates are accepted.	UT, Validation	FR-LEV-001
TS-13	Leave decision: applicant cannot decide own leave (403); non-team Manager receives 403; a decided leave returns 409; approval publishes LeaveApproved, rejection publishes LeaveRejected.	UT, IT, Authorization	FR-LEV-005, 006
TS-14	A second check-in on the same day returns 409; check-out without check-in returns 409; check-out earlier than check-in is rejected.	UT, API	FR-ATT-001, 002
TS-15	Ticket status skipping (for example Assigned to Resolved) returns 409; the full sequence Open to Closed succeeds and sets ResolvedAt.	UT, API	FR-SUP-004 to 007
TS-16	Support Staff changing status of a ticket not assigned to them receives 403; an Employee viewing another's ticket receives 403.	API, Authorization	FR-SUP-002, 005
TS-17	The same Kafka event delivered twice creates one notification.	IT	FR-NOT-001
TS-18	A user marking another user's notification as read receives 403.	API, Authorization	FR-NOT-003
TS-19	Requests without a token, with an expired token or with a tampered token receive 401 at the gateway.	API, Authorization	All
TS-20	Error responses contain no stack trace, SQL or internal host names and always include a traceId.	API, Error handling	All
TS-21	Log samples from all services contain no passwords, tokens or salary amounts, and share the correlation ID of the request.	IT, Inspection	NFR-LOG-006
TS-22	When Employee Service is unavailable, Add Salary returns 503 and stores nothing.	IT, Error handling	FR-SAL-001
TS-23	A reset token cannot be used twice or after expiry; forgot-password returns the same response for known and unknown e-mails.	UT, API	FR-AUT-002, 003
TS-24	All services are registered in Eureka and reachable through the gateway under /api/v1; an unsupported version returns 404.	IT	ARC-002, SD-001, VER-001

30. Swagger / OpenAPI Requirements

ID	Requirement
SWG-001	Every microservice shall publish an OpenAPI 3 document and Swagger UI describing all its endpoints.
SWG-002	Documents shall be versioned (v1) and shall list every path, method, parameter, request body, response and status code of the service.
SWG-003	The bearer-token security scheme shall be declared, and each operation shall show the roles permitted.
SWG-004	Request and response schemas shall use the field names of Section 17 and include validation constraints (required, length, format, enumeration).
SWG-005	The standard error structure and its status codes (400, 401, 403, 404, 409, 500, 503) shall be documented for each operation.
SWG-006	Each operation shall have a summary and description; examples shall be given for create operations.
SWG-007	Swagger UI shall be enabled in development and test environments and restricted or disabled in production.

31. Frontend Requirements

The React frontend is implemented if time allows. When implemented, it shall satisfy the requirements below and the twelve wireframe screens.

31.1 General Frontend Requirements

ID	Requirement
FE-001	The frontend shall call only the API Gateway, using versioned /api/v1 endpoints.
FE-002	All screens shall keep the wireframe structure: left navigation, top header, workspace, tables, cards, status badges and action buttons.
FE-003	Menus and action buttons shall be shown according to the signed-in role (Section 9.3). The server remains the authority (NFR-SEC-007).
FE-004	The frontend shall never store or display passwords, shall not write tokens to logs or the console, and shall send the token only to the gateway.
FE-005	On 401 the frontend shall return the user to the login screen; on 403 it shall show a permission message; on 400 it shall show field-level messages; on 503 or 500 it shall show a generic error and the traceId.
FE-006	Tables shall provide paging, and directory screens shall provide search and filters using the API's parameters.
FE-007	Delete actions shall ask for confirmation before the request is sent.
FE-008	Status values (EmploymentStatus, leave Status, overtime Status, ticket Status, notification read state) shall be shown as status badges.
FE-009	Every screen shall show loading, empty and error states.
FE-010	Salary, bonus and payslip data shall be displayed only on the salary and payslip screens to authorized users, and never in employee directory screens.

31.2 Wireframe Screen Mapping

No.	Screen	Backing requirements	Notes
1	Landing / Login	FR-AUT-001	The role preview and role selector are informational; the role always comes from the account. The registration/help link shows how to contact an administrator; there is no self-registration (accounts are created by Admin, FR-AUT-004). "Remember me" only affects how long the client keeps the session, within the token lifetime policy.
2	Forgot Password	FR-AUT-002, FR-AUT-003	Request a reset link with the registered email; set the new password from the link.
3	Admin Dashboard	FR-EMP-003, FR-LEV-004, FR-ATT-005, FR-SAL-018, FR-SUP-003, FR-NOT-002	Summary widgets are composed by the frontend from totalCount and lists returned by existing endpoints. No new backend feature is introduced (Open Item OI-06).
4	Employee Management	FR-EMP-001 to FR-EMP-006	Directory with search (FR-EMP-004), filter, View, Edit, Delete and Add Employee.

No.	Screen	Backing requirements	Notes
5	Department Management	FR-DEP-001 to FR-DEP-005	Employee count per department is the totalCount of FR-EMP-004 filtered by DepartmentId. Department lead and status are not in the specified data model (Open Item OI-06).
6	Leave Management	FR-LEV-001 to FR-LEV-006	Application form, history and pending requests. The leave-balance widget is not supported by the specified data model (Open Items OI-03, OI-06).
7	Attendance	FR-ATT-001 to FR-ATT-005	Daily, weekly and monthly hours are calculated by the frontend from CheckInTime and CheckOutTime.
8	Salary and Payroll — Employee View	FR-SAL-002, 006, 008, 011, 012, 013, 021	Read-only for salary, components, revision history, Approved bonus and payslip; the only write action is Submit Overtime Request.
9	Salary and Payroll — Admin View	FR-SAL-001 to FR-SAL-021	Available to HR and Admin. Salary revision, bonus, overtime review and approval, payroll, payslips.
10	Support	FR-SUP-001 to FR-SUP-007	Create ticket, priority, description, assign, update status, resolve, close.
11	Notifications	FR-NOT-002, FR-NOT-003	All and Unread tabs use the isRead filter; categories map to the notification Type.
12	Profile	FR-EMP-007, FR-EMP-008	Edit Profile allows FirstName, LastName and Phone only.

32. Assumptions and Constraints

32.1 Constraints

ID	Constraint
C-01	The backend uses C#, .NET and ASP.NET Core Web API with PostgreSQL and Entity Framework Core.
C-02	The architecture is microservices with an API Gateway, Eureka, Apache Kafka, database-per-service, central logging, security and API versioning.
C-03	The system has exactly the seven business microservices listed in ARC-001; there is no Project Service.
C-04	No physical foreign keys between service databases; no direct access to another service's database.
C-05	Swagger/OpenAPI is used for API documentation, Bruno for API testing and xUnit for unit testing.
C-06	No feature outside this document shall be added to the current release.

32.2 Assumptions

ID	Assumption
A-01	Every user account corresponds to exactly one employee record; EmployeeId identifies the user in all services.
A-02	Access tokens carry UserId, EmployeeId and role, so self-service requests need no extra call to identify the caller.
A-03	A Manager's team consists of the employees who report to that Manager. The requirements statement has no reporting-relationship field; Employee Service is the authority that resolves team membership (Open Item OI-01).
A-04	Enumeration values not fixed in the requirements statement are proposed here: EmploymentStatus (Active, Inactive), Priority (Low, Medium, High), attendance Status (CheckedIn, CheckedOut), bonus Status (Pending, Approved), overtime Status (Pending, Approved, Rejected), payroll Status (Generated, Finalized).
A-05	The system uses a single currency; amounts carry two decimal places.
A-06	Employee deletion is implemented so that historical records in other services remain resolvable; whether the row is physically or logically removed is a design decision.
A-07	The requirements statement names fields for Employee, Department, Leave, Attendance, Support and Notification. The Salary Service entities and fields in Section 17 are derived from the required operations.
A-08	The overtime amount for payroll is entered by HR or Admin when overtime is included; automatic rate calculation is not part of this release.
A-09	Payroll is generated per calendar month.
A-10	Employee.Role and the role of the user account use the same set of values (Admin, HR, Manager, Employee, SupportStaff) (Open Item OI-02).
A-11	The project deployment serves up to 500 employees and 100 concurrent users.

ID	Assumption
A-12	Timestamps are stored in UTC; the frontend converts them for display.
A-13	Admin's "access to administrative information" is interpreted as read-only access to leave requests in addition to the rights listed for Admin.
A-14	Because the rejection of leave records its decision maker in ApprovedBy and ApprovedAt, those fields are used for both decisions (BR-LEV-005).

32.3 Open Items

ID	Open item
OI-01	Define how team membership is stored (for example a ManagerId attribute of Employee) so that Team scope can be resolved.
OI-02	Decide how Employee.Role and the account role are kept consistent when a role changes (FR-AUT-005, FR-EMP-005).
OI-03	Define the list of LeaveType values and whether leave balances are required; balances are not part of this release.
OI-04	Define which additional support operations, if any, Admin may authorize for Managers.
OI-05	Choose how the password-reset link is delivered (for example an SMTP e-mail service); this is the only outbound integration.
OI-06	Wireframe elements without backing data in the requirements statement — leave balance, department lead, department status, dashboard analytics beyond counts — are excluded until confirmed.
OI-07	Decide whether the Security component is a module of the API Gateway or a separate identity provider (see ARC-013).

33. Future Enhancements

The following are candidates for later releases. They are not part of this release and shall not be implemented under this SRS.

- Leave balances and configurable leave policies (OI-03).
- Department lead and department status; reporting dashboards.
- Additional notification channels (e-mail, push).
- Automatic overtime rate calculation and payroll rules.
- Multi-factor authentication.
- Audit reporting screens over the logged business actions.
- Containerized deployment and orchestration for the full stack.

34. Acceptance Criteria

The system is accepted when all criteria below are met and demonstrated to the mentor.

ID	Acceptance criterion	Verified by
AC-001	All seven microservices run, are registered in Eureka and are reachable only through the API Gateway under /api/v1.	TS-24
AC-002	Every service uses its own database; schema review shows no cross-service foreign key and no cross-database access.	Schema review
AC-003	All 61 functional requirements are implemented and each has at least one passing automated test.	Appendix A
AC-004	For every role and endpoint, permitted calls succeed and forbidden calls return 403 as defined in the Role Permission Matrix.	TR-004
AC-005	An Employee cannot create or change salary, bonus or salary revision through any endpoint.	TS-05
AC-006	The leave workflow works end to end (Pending, Approved or Rejected) and produces the correct notification.	TS-12, TS-13
AC-007	The overtime workflow works end to end: submit, approve or reject, include in payroll, payslip.	TS-08 to TS-11

ID	Acceptance criterion	Verified by
AC-008	The support workflow follows Open, Assigned, InProgress, Resolved, Closed; illegal transitions return 409.	TS-15
AC-009	Attendance enforces one record per employee per day and rejects a check-out earlier than check-in.	TS-14
AC-010	The five Kafka events are published and each creates the specified notification exactly once.	TS-08, TS-17
AC-011	Swagger UI for every service documents all endpoints, errors and the bearer scheme.	Inspection
AC-012	Bruno collections for every service run from the command line and pass.	TR-003, TR-010
AC-013	xUnit tests pass and business-layer coverage is at least 70%.	NFR-TST-002
AC-014	Central logging shows one correlation ID across gateway and services; logs contain no passwords, tokens or salary amounts.	TS-21
AC-015	Passwords are stored only as hashes, and error responses expose no internal detail.	TS-20, TS-23
AC-016	The measured performance meets NFR-PERF-001 to NFR-PERF-005.	TR-009
AC-017	If the frontend is implemented, all twelve wireframe screens are available to the roles that may use them.	Demonstration
AC-018	The delivered system contains no Project Service and no feature that is absent from this SRS.	Review

35. Glossary

Term	Definition
Access token	Signed, time-limited credential issued at login that identifies the user (UserId, EmployeeId, role).
API Gateway	Single entry point that authenticates and routes client requests to microservices.
Approved bonus	A bonus whose Status is Approved; the only bonus an employee can view.
Bonus	An additional payment recorded by HR or Admin for an employee.
Bruno	API client used for API testing with collections of requests and assertions.
Correlation ID	Identifier attached to a request and to the events and calls it causes, used to trace it in logs.
Database-per-service	Pattern in which each microservice owns a private database that other services cannot access.
Dead-letter topic	Kafka topic that receives messages that could not be processed.
Entity Framework Core	Object-relational mapper used by the services to access PostgreSQL.
Eureka	Service registry used for service discovery.
Idempotent consumer	A consumer that produces the same result when the same event is delivered more than once.
Kafka	Apache Kafka, the event-streaming platform used for asynchronous events.
Microservice	Independently deployable service that owns one business capability and its data.
OpenAPI / Swagger	Specification and tooling for describing and exploring REST APIs.
Overtime	Extra hours submitted by an employee and approved before inclusion in payroll.
Payroll	Monthly run that calculates payroll items for employees.
Payslip	Per-employee statement of earnings, deductions and net pay for a payroll.
RBAC	Role-based access control: permissions are granted to roles.
REST	Architectural style for synchronous HTTP APIs.
Salary component	A named earning or deduction that is part of an employee's salary.
Salary revision	A permanent record of a change in an employee's base salary.
Service reference	An identifier (for example EmployeeId) stored in one service that refers to data owned by another; not a foreign key.
Team	The employees who report to a given Manager (Assumption A-03).
xUnit	Unit-testing framework for .NET.

Appendix A — Functional Requirement Traceability Matrix

Verification codes: UT = unit test (xUnit); API = Bruno API test; IT = integration test; AUTH = authorization test; VAL = validation test; ERR = error-handling test.

FR	Requirement	Endpoint(s)	Business rules	Verification
FR-AUT-001	Login	POST /api/v1/auth/login	BR-GEN-001	UT, API, VAL, ERR
FR-AUT-002	Request Password Reset	POST /api/v1/auth/forgot-password	–	UT, API, VAL, ERR
FR-AUT-003	Reset Password	POST /api/v1/auth/reset-password	–	UT, API, VAL, ERR
FR-AUT-004	Manage User Accounts	POST /api/v1/users GET /api/v1/users GET /api/v1/users/{userId} PUT /api/v1/users/{userId} PUT /api/v1/users/{userId}/deactivate	BR-GEN-001	UT, API, AUTH, VAL, ERR
FR-AUT-005	Assign User Role	PUT /api/v1/users/{userId}/role	BR-EMP-007	UT, API, AUTH, VAL, ERR
FR-EMP-001	Add Employee	POST /api/v1/employees	BR-EMP-001, BR-EMP-002, BR-EMP-003, BR-EMP-004, BR-EMP-007, BR-EMP-008, BR-GEN-004	UT, API, AUTH, VAL, ERR
FR-EMP-002	View Employee	GET /api/v1/employees/{employeeId}	BR-EMP-005, BR-GEN-002	UT, API, AUTH, VAL, ERR
FR-EMP-003	View All Employees	GET /api/v1/employees	–	UT, API, AUTH, VAL, ERR
FR-EMP-004	Search Employees	GET /api/v1/employees/search	BR-EMP-005, BR-GEN-002	UT, API, AUTH, VAL, ERR
FR-EMP-005	Update Employee	PUT /api/v1/employees/{employeeId}	BR-EMP-001, BR-EMP-003, BR-EMP-004, BR-EMP-007, BR-EMP-008, BR-EMP-009, BR-GEN-004	UT, API, AUTH, VAL, ERR
FR-EMP-006	Delete Employee	DELETE /api/v1/employees/{employeeId}	BR-GEN-003	UT, API, AUTH, VAL, ERR
FR-EMP-007	View Own Profile	GET /api/v1/employees/me	BR-EMP-005, BR-GEN-006	UT, API, AUTH, VAL, ERR
FR-EMP-008	Update Own Profile	PUT /api/v1/employees/me	BR-EMP-003, BR-EMP-006, BR-GEN-004, BR-GEN-006	UT, API, AUTH, VAL, ERR
FR-EMP-009	View Team Employees	GET /api/v1/employees/team	BR-EMP-005, BR-GEN-002	UT, API, AUTH, VAL, ERR
FR-DEP-001	Add Department	POST /api/v1/departments	BR-DEP-001, BR-DEP-003, BR-GEN-004	UT, API, AUTH, VAL, ERR
FR-DEP-002	View Department	GET /api/v1/departments/{departmentId}	–	UT, API, AUTH, VAL, ERR
FR-DEP-003	View All Departments	GET /api/v1/departments	–	UT, API, AUTH, VAL, ERR

FR	Requirement	Endpoint(s)	Business rules	Verification
FR-DEP-004	Update Department	PUT /api/v1/departments/{departmentId}	BR-DEP-001, BR-DEP-003, BR-GEN-004	UT, API, AUTH, VAL, ERR
FR-DEP-005	Delete Department	DELETE /api/v1/departments/{departmentId}	BR-DEP-002	UT, API, AUTH, VAL, ERR
FR-SAL-001	Add Salary	POST /api/v1/salaries	BR-GEN-004, BR-SAL-001, BR-SAL-002, BR-SAL-004, BR-SAL-007	UT, API, AUTH, VAL, ERR
FR-SAL-002	View Salary	GET /api/v1/salaries/me GET /api/v1/salaries/{salaryId}	BR-SAL-003, BR-SAL-014	UT, API, AUTH, VAL, ERR
FR-SAL-003	Update Salary	PUT /api/v1/salaries/{salaryId}	BR-GEN-004, BR-SAL-001, BR-SAL-002, BR-SAL-007	UT, API, AUTH, VAL, ERR
FR-SAL-004	Add Salary Component	POST /api/v1/salaries/{salaryId}/components	BR-SAL-001, BR-SAL-002, BR-SAL-007	UT, API, AUTH, VAL, ERR
FR-SAL-005	Update Salary Component	PUT /api/v1/salaries/{salaryId}/components/{componentId}	BR-GEN-004, BR-SAL-001, BR-SAL-002, BR-SAL-007	UT, API, AUTH, VAL, ERR
FR-SAL-006	View Salary Components	GET /api/v1/salaries/me/components GET /api/v1/salaries/{salaryId}/components	BR-SAL-003, BR-SAL-014	UT, API, AUTH, VAL, ERR
FR-SAL-007	Create Salary Revision	POST /api/v1/salaries/{salaryId}/revisions	BR-SAL-001, BR-SAL-002, BR-SAL-005, BR-SAL-006, BR-SAL-007	UT, API, AUTH, VAL, ERR
FR-SAL-008	View Salary Revision History	GET /api/v1/salaries/me/revisions GET /api/v1/salaries/{salaryId}/revisions	BR-SAL-003, BR-SAL-005, BR-SAL-014	UT, API, AUTH, VAL, ERR
FR-SAL-009	Add Bonus	POST /api/v1/salaries/bonuses	BR-GEN-004, BR-SAL-001, BR-SAL-002, BR-SAL-007, BR-SAL-013	UT, API, AUTH, VAL, ERR
FR-SAL-010	Update Bonus	PUT /api/v1/salaries/bonuses/{bonusId}	BR-GEN-004, BR-SAL-001, BR-SAL-002, BR-SAL-007, BR-SAL-013	UT, API, AUTH, VAL, ERR
FR-SAL-011	View Bonus	GET /api/v1/salaries/bonuses	BR-SAL-003, BR-SAL-013, BR-SAL-014	UT, API, AUTH, VAL, ERR
FR-SAL-012	Submit Overtime Request	POST /api/v1/salaries/overtime	BR-GEN-004, BR-GEN-006, BR-SAL-008	UT, API, AUTH, VAL, ERR
FR-SAL-013	View Overtime Requests	GET /api/v1/salaries/overtime	BR-GEN-002, BR-SAL-003	UT, API, AUTH, VAL, ERR
FR-SAL-014	Approve Overtime	PUT /api/v1/salaries/overtime/{overtimeId}/approve	BR-GEN-005, BR-SAL-009	UT, API, AUTH, VAL, ERR
FR-SAL-015	Reject Overtime	PUT /api/v1/salaries/overtime/{overtimeId}/reject	BR-GEN-005, BR-SAL-009	UT, API, AUTH, VAL, ERR
FR-SAL-016	Include Approved Overtime in Payroll	POST /api/v1/salaries/payrolls/{payrollId}/overtime	BR-SAL-001, BR-SAL-007, BR-SAL-010	UT, API, AUTH, VAL, ERR
FR-SAL-017	Generate Payroll	POST /api/v1/salaries/payrolls	BR-SAL-010, BR-SAL-011, BR-SAL-013	UT, API, AUTH, VAL, ERR

FR	Requirement	Endpoint(s)	Business rules	Verification
FR-SAL-018	View Payroll	GET /api/v1/salaries/payrolls GET /api/v1/salaries/payrolls/{payrollId}	BR-SAL-014	UT, API, AUTH, VAL, ERR
FR-SAL-019	View Payroll Items	GET /api/v1/salaries/payrolls/{payrollId}/items	BR-SAL-014	UT, API, AUTH, VAL, ERR
FR-SAL-020	Generate Payslip	POST /api/v1/salaries/payrolls/{payrollId}/payslips	BR-SAL-012, BR-SAL-014	UT, API, AUTH, VAL, ERR
FR-SAL-021	View Payslip	GET /api/v1/salaries/payslips GET /api/v1/salaries/payslips/{payslipId}	BR-SAL-003, BR-SAL-014	UT, API, AUTH, VAL, ERR
FR-LEV-001	Apply Leave	POST /api/v1/leaves	BR-GEN-006, BR-LEV-001, BR-LEV-002, BR-LEV-004, BR-LEV-006	UT, API, AUTH, VAL, ERR
FR-LEV-002	View Leave	GET /api/v1/leaves/{leavedId}	BR-GEN-002, BR-LEV-004	UT, API, AUTH, VAL, ERR
FR-LEV-003	View Leave History	GET /api/v1/leaves/history	BR-GEN-006, BR-LEV-004	UT, API, AUTH, VAL, ERR
FR-LEV-004	View Leave Requests	GET /api/v1/leaves/requests	BR-GEN-002	UT, API, AUTH, VAL, ERR
FR-LEV-005	Approve Leave	PUT /api/v1/leaves/{leavedId}/approve	BR-GEN-005, BR-LEV-003, BR-LEV-005	UT, API, AUTH, VAL, ERR
FR-LEV-006	Reject Leave	PUT /api/v1/leaves/{leavedId}/reject	BR-GEN-005, BR-LEV-003, BR-LEV-005	UT, API, AUTH, VAL, ERR
FR-ATT-001	Check In	POST /api/v1/attendance/check-in	BR-ATT-001, BR-ATT-003, BR-ATT-004, BR-GEN-006	UT, API, AUTH, VAL, ERR
FR-ATT-002	Check Out	POST /api/v1/attendance/check-out	BR-ATT-002, BR-ATT-003, BR-ATT-004, BR-GEN-004	UT, API, AUTH, VAL, ERR
FR-ATT-003	View Attendance	GET /api/v1/attendance/me	BR-ATT-005, BR-GEN-006	UT, API, AUTH, VAL, ERR
FR-ATT-004	View Attendance History	GET /api/v1/attendance/history	BR-ATT-005	UT, API, AUTH, VAL, ERR
FR-ATT-005	View Team Attendance	GET /api/v1/attendance/team	BR-ATT-005, BR-GEN-002	UT, API, AUTH, VAL, ERR
FR-SUP-001	Create Support Ticket	POST /api/v1/support-tickets	BR-GEN-006, BR-SUP-001, BR-SUP-002	UT, API, AUTH, VAL, ERR
FR-SUP-002	View Ticket	GET /api/v1/support-tickets/{ticketId}	BR-SUP-004, BR-SUP-005	UT, API, AUTH, VAL, ERR
FR-SUP-003	View Tickets	GET /api/v1/support-tickets	BR-SUP-004, BR-SUP-005	UT, API, AUTH, VAL, ERR
FR-SUP-004	Assign Ticket	PUT /api/v1/support-tickets/{ticketId}/assign	BR-SUP-003, BR-SUP-005, BR-SUP-006	UT, API, AUTH, VAL, ERR

FR	Requirement	Endpoint(s)	Business rules	Verification
FR-SUP-005	Update Ticket Status	PUT /api/v1/support-tickets/{ticketId}/status	BR-SUP-003, BR-SUP-005	UT, API, AUTH, VAL, ERR
FR-SUP-006	Resolve Ticket	PUT /api/v1/support-tickets/{ticketId}/resolve	BR-SUP-003, BR-SUP-005, BR-SUP-007	UT, API, AUTH, VAL, ERR
FR-SUP-007	Close Ticket	PUT /api/v1/support-tickets/{ticketId}/close	BR-SUP-003, BR-SUP-005	UT, API, AUTH, VAL, ERR
FR-NOT-001	Create Notification from Event	Kafka consumer (no REST endpoint)	BR-NOT-001, BR-NOT-002, BR-NOT-003	UT, IT, ERR
FR-NOT-002	View Notifications	GET /api/v1/notifications	BR-GEN-006, BR-NOT-001	UT, API, AUTH, VAL, ERR
FR-NOT-003	Mark Notification as Read	PUT /api/v1/notifications/{notificationId}/read	BR-NOT-001, BR-NOT-004	UT, API, AUTH, VAL, ERR